

在图10.9中,  $d_{\min} > 2t$ ; 因为下一个整数增量是1, 所以可以说  $d_{\min} = 2t + 1$ 。

为了保证纠正所有情况下最多  $t$  个差错, 块编码中的最小汉明距离是  $d_{\min} = 2t + 1$ 。

### 例10.9

一个编码方案有汉明距离  $d_{\min} = 4$ 。这个方案的检错和纠错能力是多少?

解:

这个方案保证检测到最多3个差错 ( $s=3$ ), 但是它能纠正1个差错。换言之, 如果这个编码用于纠错, 它的部分能力被浪费了。纠错编码需要的最小距离是奇数 (3、5、7、...)。

## 10.3 线性块编码

目前, 几乎所有使用的块编码都属于一个称为线性块编码 (linear block code) 的子集。用于检错和纠错的非线性块编码的应用不是很广泛, 因为它们的结构使得理论分析和实现都很困难。因此, 我们只介绍线性块编码。

线性块编码的正式定义需要抽象代数 (尤其是Galois领域) 理论, 这超出了本书范围。所以我们给出一个非正式定义。为此目的, 线性块编码是一种由任何两个有效码字的异或 (模2加法) 产生的另一个有效码字。

在线性块编码中, 任两个有效码字的异或 (XOR) 生成另一个有效码字。

### 例10.10

让我们观察定义在表10.1和表10.2中的两个编码是否属于线性块编码。

1. 表10.1中的方案是线性块编码, 因为任何一个码字和任何另一个码字XOR的结果是一个有效码字。例如, 第2个和第3个码字的XOR得到第4个码字。

2. 表10.2中的方案也是线性块编码。我们可以通过异或两个其他码字得到所有的4个码字。

### 10.3.1 线性块编码的最小距离

可以简单地求出线性块编码的最小汉明距离。最小汉明距离是具有最小1的个数的非0有效码字中1的个数。

### 例10.11

在第一个编码方案 (表10.1) 中, 非零码字的1的个数是2、2和2。因此, 最小汉明距离是  $d_{\min}=2$ 。在第二个编码方案 (表10.2) 中, 非零码字的1的个数是3、3和4。因此  $d_{\min}=3$ 。

### 10.3.2 一些线性块编码

现在给出一些线性块编码。这些代码是线性块编码不证自明, 因为我们可以轻易地得到代码和译码算法并检验它们的性能。

#### 简单奇偶校验编码

可能最熟悉的差错检测编码是简单奇偶校验编码 (simple parity-check code)。在这个编码中,  $k$  位数据字变成  $n$  位码字, 这里  $n=k+1$ 。选择称为奇偶位的额外位, 使得码字中1的总个数为偶数。虽然另一些实现规定1的个数为奇数, 但是我们讨论偶数情况。这种编码的最小汉明距离是  $d_{\min}=2$ , 这意味着编码是单个位检错编码, 它不能纠正任何差错。

简单奇偶校验编码是  $n = k + 1$ 、 $d_{\min} = 2$  的单个位检错编码。



我们的第一个编码（表10.1）是 $k=2$ 、 $n=3$ 的奇偶校验编码。表10.3中的编码也是奇偶校验编码，这里 $k=4$ 、 $n=5$ 。

图10.10说明（在发送方）编码器和（在接收方）译码器的可能结构。编码器使用一个生成器，该生成器得到一个4位数据字的副本（ $a_0$ 、 $a_1$ 、 $a_2$ 和 $a_3$ ）并产生一个奇偶位 $r_0$ 。数据字的各个位和奇偶位（parity bit）组成一个5位码字。增加的奇偶位使得码字中的1的个数为偶数。

表10.3 简单奇偶校验码C (5, 4)

| 数据字  | 码 字   | 数据字  | 码 字   |
|------|-------|------|-------|
| 0000 | 00000 | 1000 | 10001 |
| 0001 | 00011 | 1001 | 10010 |
| 0010 | 00101 | 1010 | 10100 |
| 0011 | 00110 | 1011 | 10111 |
| 0100 | 01001 | 1100 | 11000 |
| 0101 | 01010 | 1101 | 11011 |
| 0110 | 01100 | 1110 | 11101 |
| 0111 | 01111 | 1111 | 11110 |

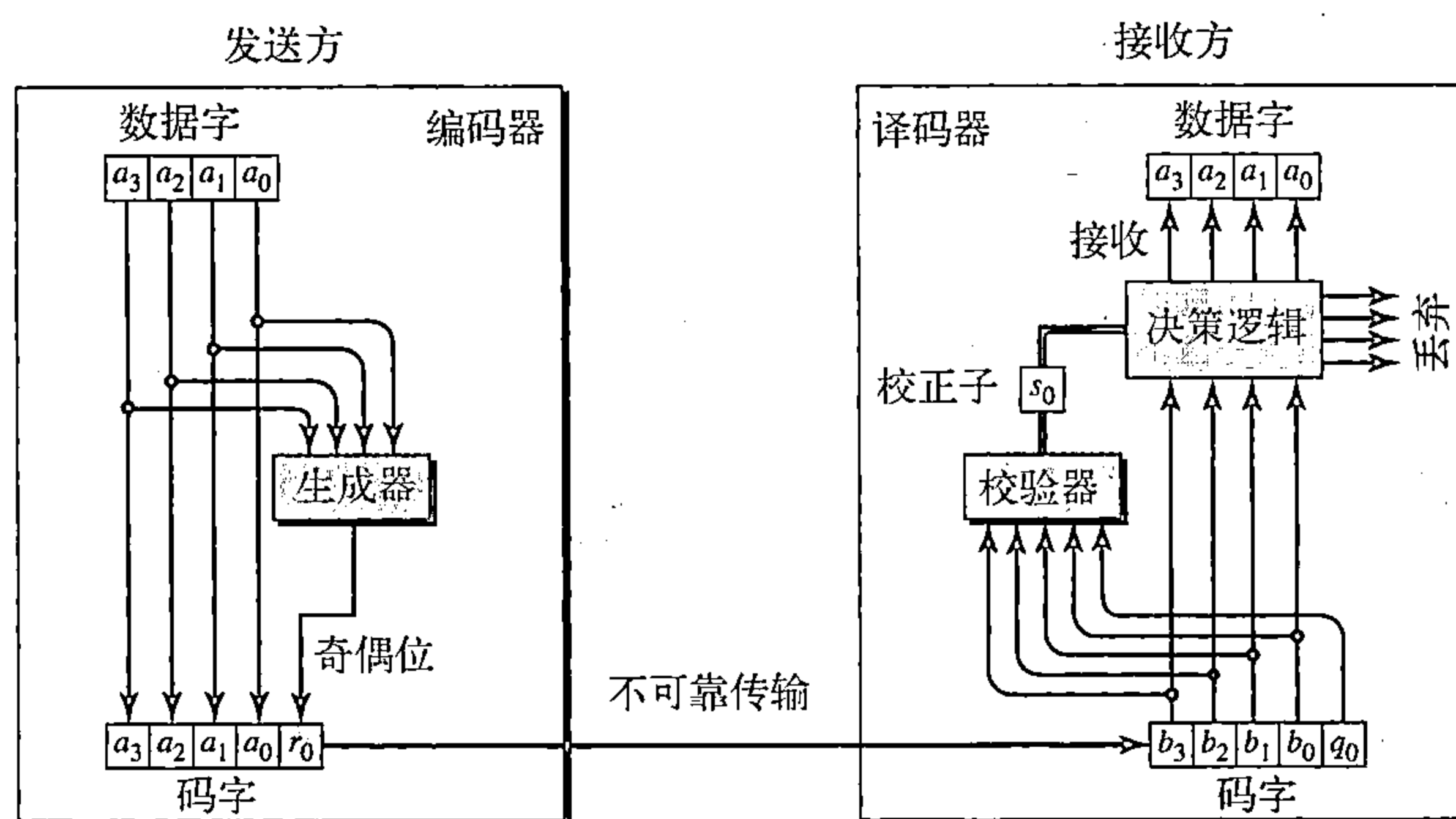


图10.10 简单奇偶校验编码的编码器和译码器解码器

通常这是通过数据字的4位做加法（模2运算）实现；结果是奇偶位的值。换言之，

$$r_0 = a_3 + a_2 + a_1 + a_0 \quad (\text{模2运算})$$

如果1的个数是偶数，结果是0；如果1的个数是奇数，结果是1。在这两种情况中，码字中1的总个数是偶数。

发送方发送一个码字，该码字可能在传输中被破坏。接收方接收到一个5位码字。接收方的校验器将做与发送方的生成器相同的工作，不同的是所有的5位都要做加法。称为校正子（syndrome），只有1位。当接收到的码字中1的个数是偶数时，校正子是0，否则它是1。

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \quad (\text{模2运算})$$

将校正子传递给决策逻辑分析器。如果校正子是0，接收到的码字中没有差错；其中的数据部分作为数据字接收。如果校正子是1，接收到的码字中的数据部分被丢弃。没有数据字生成。

#### 例10.12

让我们看一些传输实况。假设发送方发送数据字1011。从这个数据字生成的码字是10111，



它发送给接收方。我们检查五种情况：

1. 没有差错发生，接收到的码字是10111。校正子是0，生成数据字1011。
2. 一个单个位差错改变了 $a_1$ ，接收到的码字是100 $\bar{1}$ 1。校正子是1，没有数据字生成。
3. 一个单个位差错改变了 $r_0$ ，接收到的码字是10110。校正子是1，没有数据字生成。注意：虽然没有数据字位被破坏，但是因为编码不够复杂不能说明破坏位的位置，因此仍然没有数据字生成。
4. 一个差错改变了 $r_0$ ，第二个差错改变了 $a_3$ ，接收到的码字是00110。校正子是0，在接收方生成数据字0011。注意：这里因为校正子生成错误的数据字，简单奇偶校验译码器不能检测出偶数个差错。差错互相抵消使得校正子为0。
5. 三个位 $a_3$ 、 $a_2$ 和 $a_1$ 被差错改变，接收到的码字是01011。校正子是1，没有数据字生成。这说明简单奇偶校验除了保证检测出一个单个位差错外，还能发现任何奇数个差错。

简单奇偶校验编码能检测出奇数个差错。

更好的办法是两维奇偶校验 (two-dimensional parity check)。在这一方法中，数据字以表格形式 (行和列) 组织。在图10.11中，要发送的数据 (5个7位字节) 分别放在各自单独的行中。对于每一行和每一列，计算出一个奇偶校验位，然后将整个表发送给接收方，接收方将分别得出每一行和每一列的校正子。正如图10.11所示，两维奇偶校验能检测出表中任何位置发生的最多三个差错 (箭头指向生成的非零校正子位置)。但是，4位的差错无法检测到。

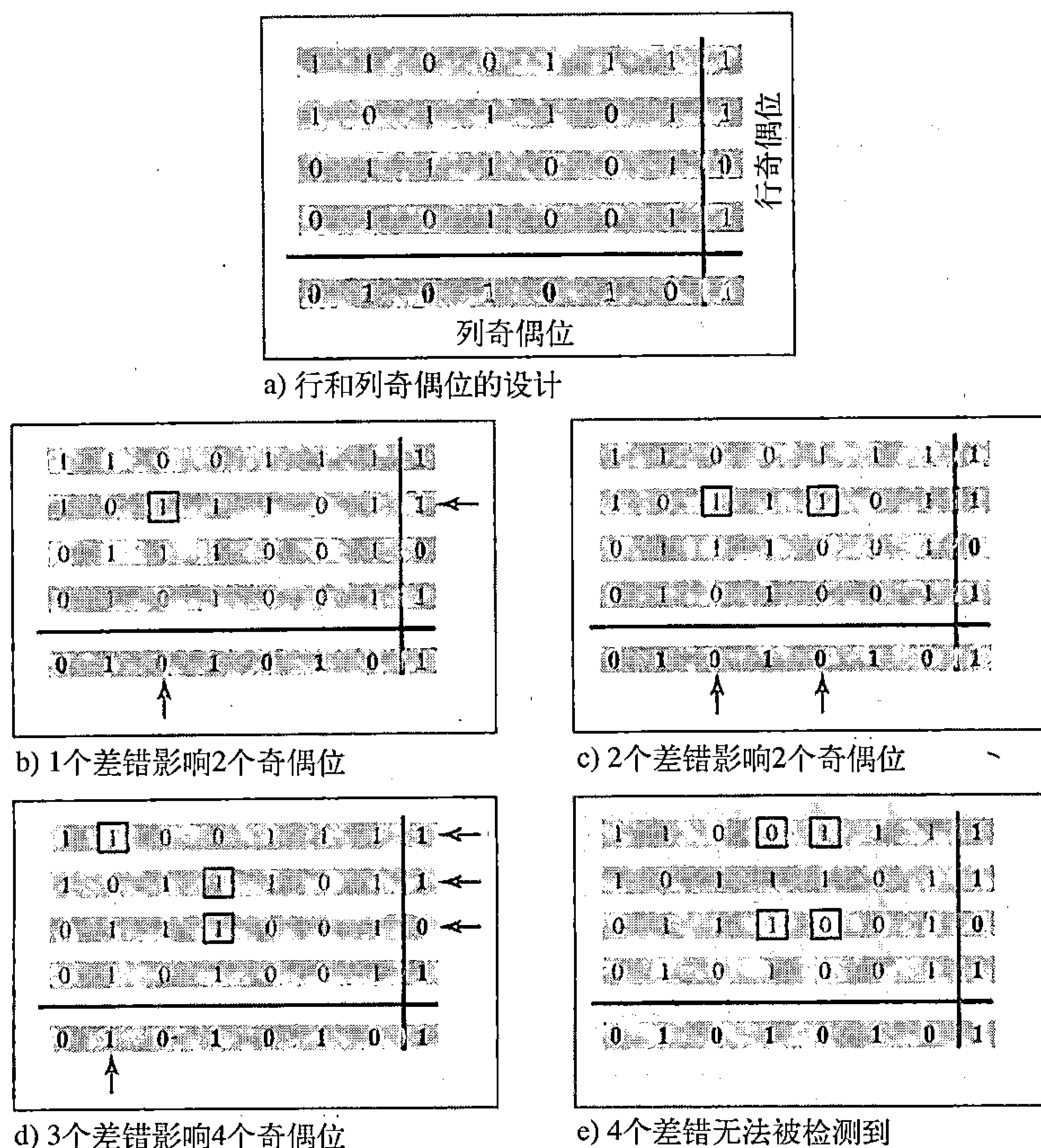


图10.11 两维奇偶校验编码



## 汉明编码

现在让我们讨论称为汉明编码 (Hamming code) 的纠错编码。这些编码最初设计是  $d_{\min} = 3$ , 这意味着它们能做最多检测出2个差错和最多纠正1个差错。虽然有一些汉明编码能纠正多于1个差错, 但是我们的讨论只限于单个位纠错编码。

让我们先找出汉明编码中  $n$  和  $k$  的关系。我们需要选择  $m \geq 3$  的整数。然后从  $m$  计算出  $n$  和  $k$ , 分别为  $n = 2^m - 1$  和  $k = n - m$ 。校验位个数  $r = m$ 。

本书讨论的所有汉明编码都有  $d_{\min} = 3$ , 这些编码中  $m$  和  $n$  之间的关系是  $n = 2^m - 1$ 。

例如, 如果  $m = 3$ , 那么  $n = 7$ ,  $k = 4$ 。这就是  $d_{\min} = 3$  的汉明编码  $C(7, 4)$ 。表10.4给出了这个编码的数据字和码字。

表10.4 汉明编码  $C(7, 4)$

| 数据字  | 码字      | 数据字  | 码字      |
|------|---------|------|---------|
| 0000 | 0000000 | 1000 | 1000110 |
| 0001 | 0001101 | 1001 | 1001011 |
| 0010 | 0010111 | 1010 | 1010001 |
| 0011 | 0011010 | 1011 | 1011100 |
| 0100 | 0100011 | 1100 | 1100101 |
| 0101 | 0101110 | 1101 | 1101000 |
| 0110 | 0110100 | 1110 | 1110010 |
| 0111 | 0111001 | 1111 | 1111111 |

图10.12给出了这个例子的编码器和译码器结构。

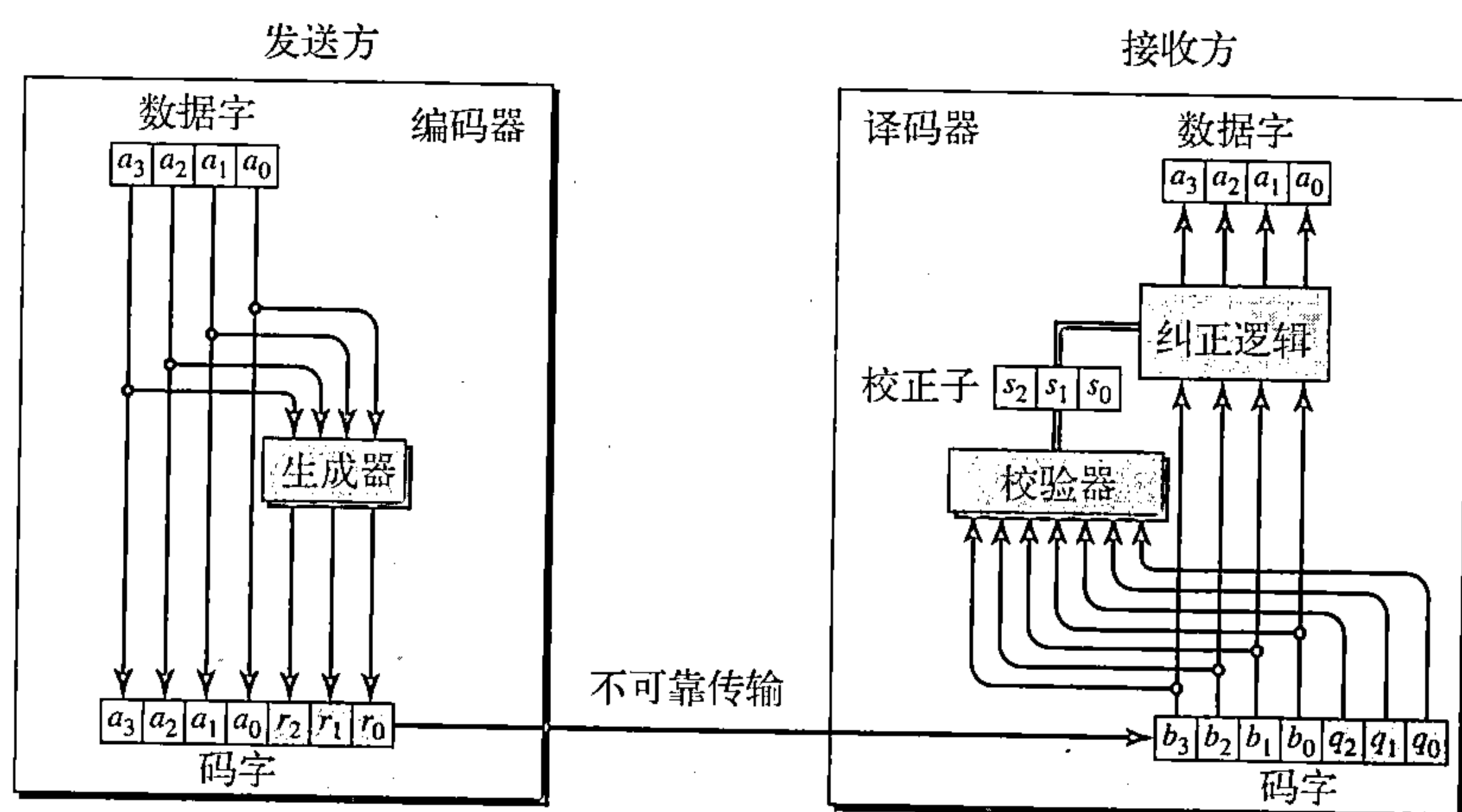


图10.12 汉明编码的编码器和译码器结构

4位数据字的副本进入生成器, 产生三个奇偶校验位  $r_0$ 、 $r_1$  和  $r_2$ , 如下所示:

$$r_0 = a_2 + a_1 + a_0 \quad (\text{模2运算})$$

$$r_1 = a_3 + a_2 + a_1 \quad (\text{模2运算})$$

$$r_2 = a_1 + a_0 + a_3 \quad (\text{模2运算})$$

换言之, 每个奇偶校验位处理4位数据字中的3位。每个4位组合 (3个数据字位和1个奇偶位) 中的1的个数一定是偶数。三个等式不是唯一的, 只要数据字4位中取3位生成独立等式 (两个等式组合不能生成第三个等式) 的任何三个等式是有效的。



$$s_0 = b_2 + b_1 + b_0 + q_0 \text{ (模2运算)}$$

$$s_1 = b_3 + b_2 + b_1 + q_1 \text{ (模2运算)}$$

$$s_2 = b_1 + b_0 + b_3 + q_2 \text{ (模2运算)}$$

校验器使用的等式与生成器使用的一样，奇偶校验位加到等式的右边。3位校正子产生8个不同的位模式（000到111），这能表示8种不同的情况。这些情况定义了接收到的码字中无差错或者7位中有1位有差错，如表10.5所示。

表10.5 由译码器的纠错逻辑分析器做出的逻辑决定

| 校正子 | 000 | 001   | 010   | 011   | 100   | 101   | 110   | 111   |
|-----|-----|-------|-------|-------|-------|-------|-------|-------|
| 差错  | 无   | $q_0$ | $q_1$ | $b_2$ | $q_2$ | $b_0$ | $b_3$ | $b_1$ |

注意：生成器不关心表10.5阴影中的四种情况（无差错或奇偶位的差错）。在其他四种情况中，4位中相应的位必须反转（0变成1或1变成0）得出正确的码字。

表10.5中的校正子的几个值是每个校正子值的组合。例如，如果 $q_0$ 发生差错， $s_0$ 是唯一受到影响的位，因此校正子是001。如果 $b_2$ 发生差错， $s_0$ 和 $s_1$ 都受到影响，因此校正子是011。类似地，如果 $b_1$ 发生差错，所有三个校正子位都受到影响，因此校正子是111。

有两点需要在这里强调一下。第一点，如果在传输中发生两个差错，生成的数据字可能不是正确的数据字。第二点，如果想用上面的编码用于纠错，还需要独特的设计。

#### 例10.13

让我们跟踪3个数据字从发送方到目的地的路径。

1. 数据字0100变成码字0100011。接收到码字0100011。校正子是000（无差错），最后的数据字是0100。

2. 数据字0111变成码字0111001。接收到码字0011001。校正子是011。根据表10.5， $b_2$ 发生差错。反转 $b_2$ 后（0变成1），最后的数据字是0111。

3. 数据字1101变成码字1101000。接收到码字0001000（两个差错）。校正子是101，这意味着 $b_0$ 发生差错。反转 $b_0$ 后，我们得到0000，这是错误的数据字。这说明我们的编码检测不到两个差错。

#### 例10.14

我们需要至少7位的数据字。计算满足这个条件的 $k$ 值和 $n$ 值。

解：

我们要求 $k = n - m$ 大于或等于7，或者 $2^m - 1 - m \geq 7$ 。

1. 如果我们设 $m=3$ ，结果是 $n=2^3-1$ 、 $k=7-3=4$ ，这是不能接受的。

2. 如果我们设 $m=4$ ，那么 $2^4-1=15$ 、 $k=15-4=11$ ，这满足了条件。因此编码是 $C(15, 11)$ 。有一些方法使得数据字为特定长度，但是这个问题的讨论和实现超出了本书的范围。

#### 性能

汉明编码只能纠正单个差错或检测出两个差错。但是，有方法使它能检测出突发差错，如图10.13所示。

关键思想是分离若干个码字间的突发性差错为每个码字只有一个差错。在数据通信中，我们通常发送数据分组或数据帧。为了让汉明编码能对长度为 $N$ 的突发差错起作用，需要从帧生成 $N$ 个码字。然后，不是每次发送一个码字，而是以表格形式组织码字，并每次发送表中一列的位。在图10.13中，以逐列发送位（从左边开始）。在每一列中，从底到顶发送位。在这种方式中，一个帧由4个码字组成，并发送给接收方。图10.13说明长度为4的突发差错破坏了



这个帧，每个码字只有1位被破坏。每个码字被破坏的位在接收方可以轻易地被纠正。

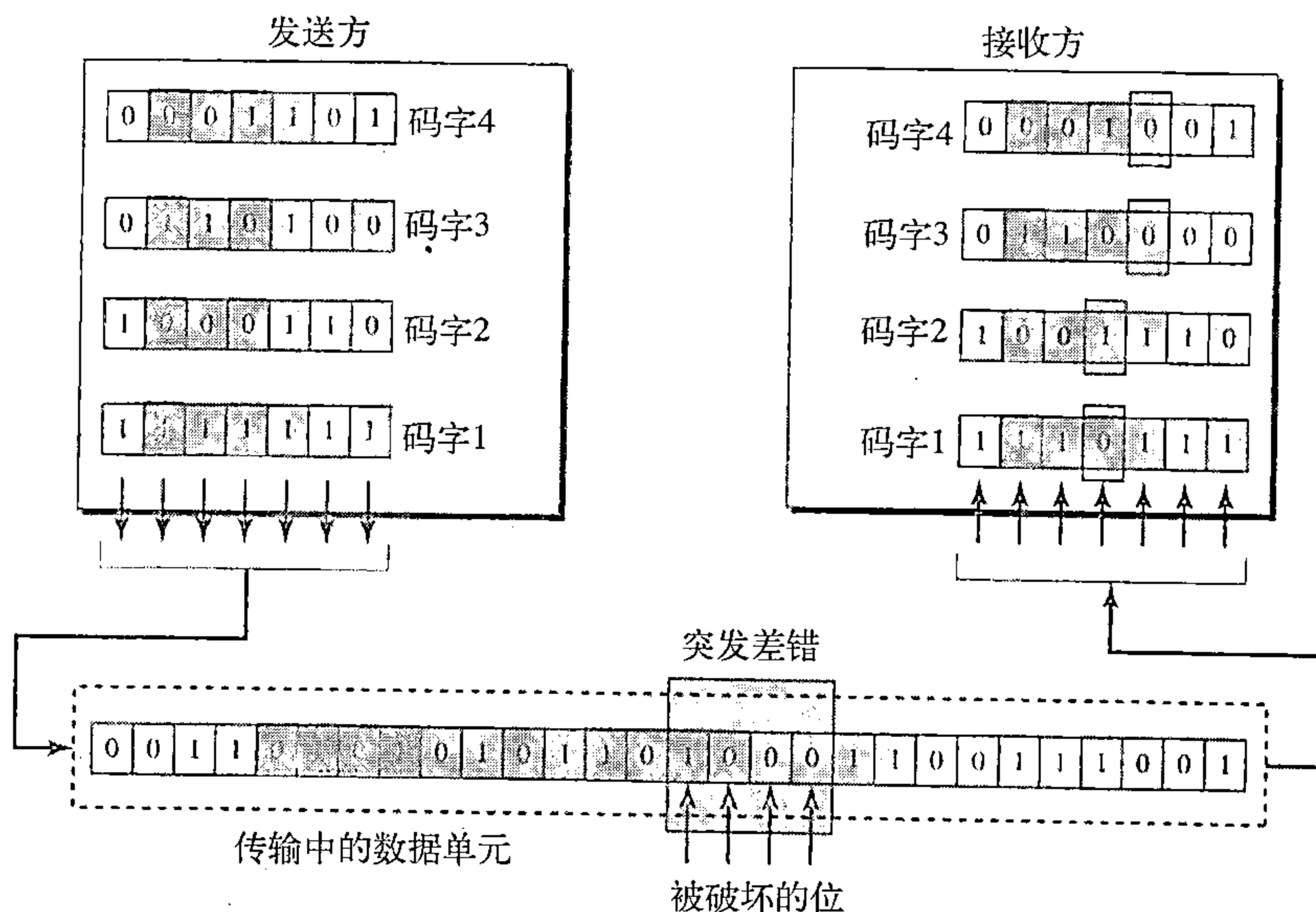


图10.13 使用汉明编码的突发差错纠正

## 10.4 循环编码

循环编码是有一个附加性质的特殊线性块编码。在循环编码 (cyclic code) 中，如果码字是循环移位 (旋转)，则其结果是另一个码字。例如，如果1011000是一个码字，循环左移，那么0110001也是一个码字。在这种情况下，如果称第一个码字中的位为 $a_0$ 到 $a_6$ ，那么称第二个码字中的位为 $b_0$ 到 $b_6$ ，我们可以通过如下方式进行移位：

$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

在最右的等式中，第一个码字中的最后一位变成了第二个码字中的第一位。

### 10.4.1 循环冗余校验

我们能够生成循环编码来纠错。但是，所需的理论背景超出了本书的范围。在本节中，我们简单地讨论一种称为循环冗余校验 (cyclic redundancy check, CRC) 的循环编码，它用于诸如LAN和WAN的网络中。图10.14说明了编码器和译码器的一种可能设计。

表10.6给出了CRC编码的一个例子。可以看到这个编码具有的线性特性和循环特性。

表10.6 C (7, 4)的CRC编码

| 数据字  | 码字      | 数据字  | 码字      |
|------|---------|------|---------|
| 0000 | 0000000 | 1000 | 1000101 |
| 0001 | 0001011 | 1001 | 1001110 |
| 0010 | 0010110 | 1010 | 1010011 |
| 0011 | 0011101 | 1011 | 1011000 |
| 0100 | 0100111 | 1100 | 1100010 |
| 0101 | 0101100 | 1101 | 1101001 |
| 0110 | 0110001 | 1110 | 1110100 |
| 0111 | 0111010 | 1111 | 1111111 |



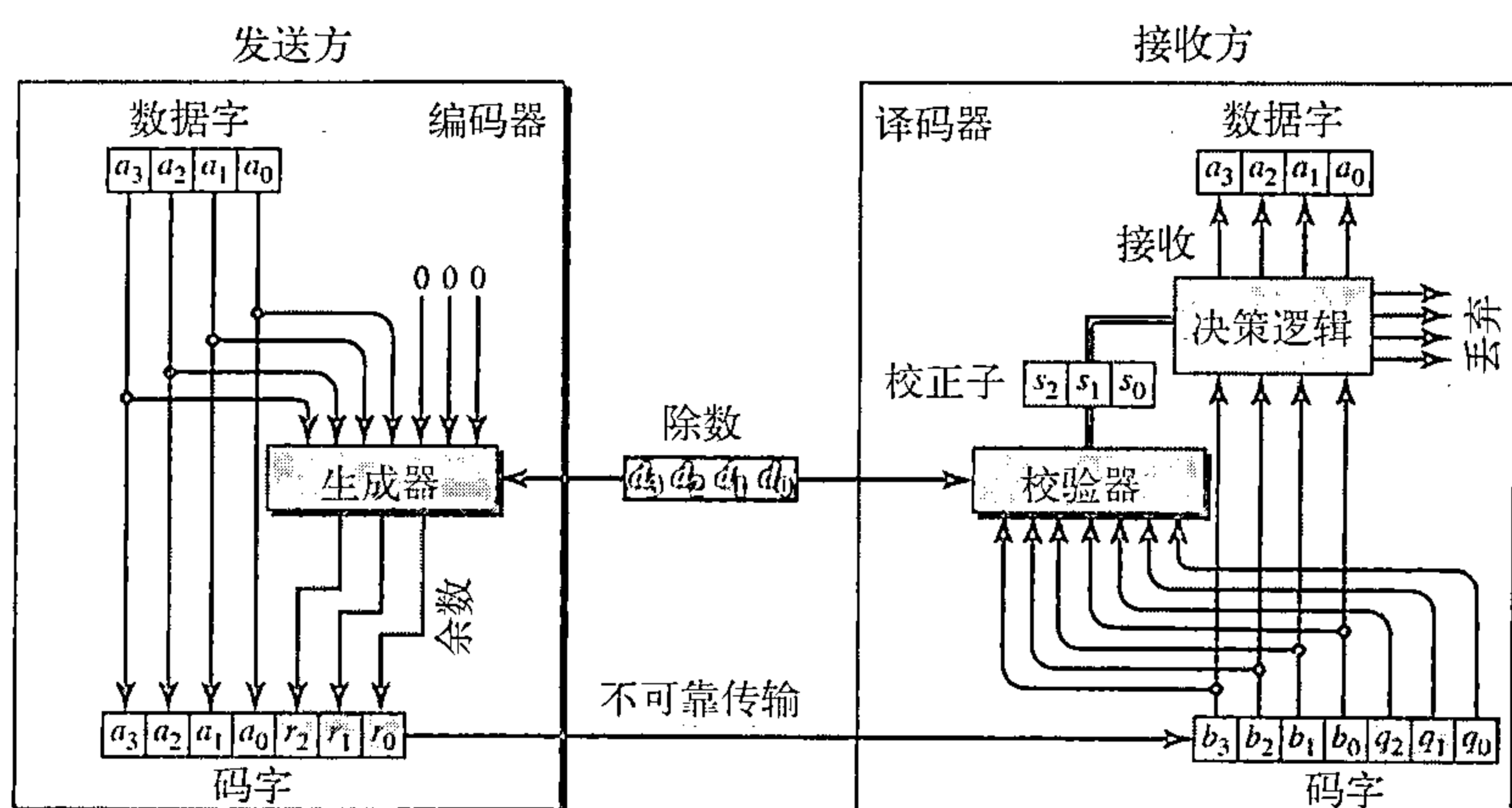


图10.14 说明了编码器和译码器的一种可能设计

在编码器中，数据字有 $k$ 位（这里 $k$ 是4）；码字有 $n$ 位（这里 $n$ 是7）。在数据字的右边加上 $n-k$ （这里是3）个位，长度加3。 $n$ 位结果传给生成器。生成器使用长度为 $n-k+1$ （这里是4）的除数，这个除数是预定义并经双方同意的。生成器用除数除增加后的数据字（模2除法）。除法的商被丢弃，余数（ $r_2 r_1 r_0$ ）加到数据字上生成码字。

译码器接收到可能被破坏的码字。全 $n$ 位的副本传递给校验器，它是生成器的复制品。校验器产生的余数是 $n-k$ （这里是3）位的校正子，它被传递给决策逻辑分析器。分析器是一个简单的函数。如果校正子全0，码字最左边的4位被接收为数据字（被认为无差错）；否则，这4位被丢弃（有差错）。

### 编码器

让我们仔细考察编码器。编码器将得到的数据字加上 $n-k$ 个0，然后它被除数去除，如图10.15所示。

模2二进制除法过程与我们熟悉的用于十进制数字的除法过程是一样的。但是，正如在本章开始所提到的，此时加法和减法是一样的。我们使用异或操作来做加法和减法。

像在十进制除法中一样，处理过程逐步完成。在每一步，除数的副本与被除数的4位进行异或操作。异或运算的结果（余数）是3位（在这种情况下），1个额外位移下来加上去组成4位进行下一步。在这种除法中有重要的一点需要记住：如果被除数（用于每一步的部分）的最左边的位是0，这一步就不能使用正常的除数，需要使用全0的除数。

当没有可移下来的位时，就得到了结果。3位余数形成了校验位（ $r_2$ 、 $r_1$ 和 $r_0$ ）。它们附加在数据字上生成码字。

### 译码器

码字可能会在传输中改变。译码器执行与编码器相同的除法处理。除法的余数是校正子。

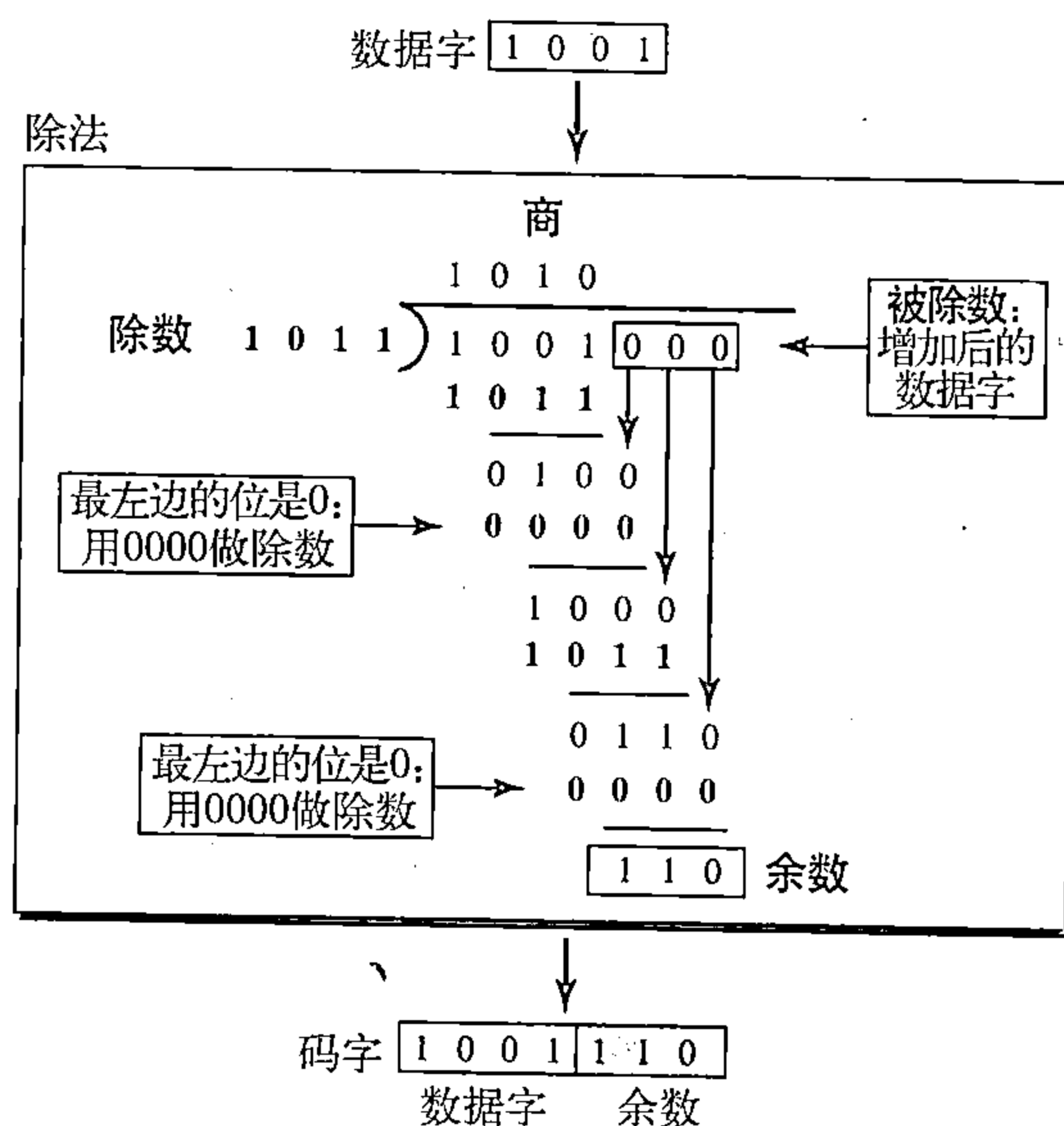


图10.15 CRC编码器中的除法



如果校正子是全0，就没有差错；数据字从接收到的码字中分离出来并被接收。否则，整个码字被丢弃。图10.16说明了两种情形：左边的图说明当没有差错发生时的校正子，其为000。右边的图说明有单个差错发生的情形，校正子不是全0（它是011）。

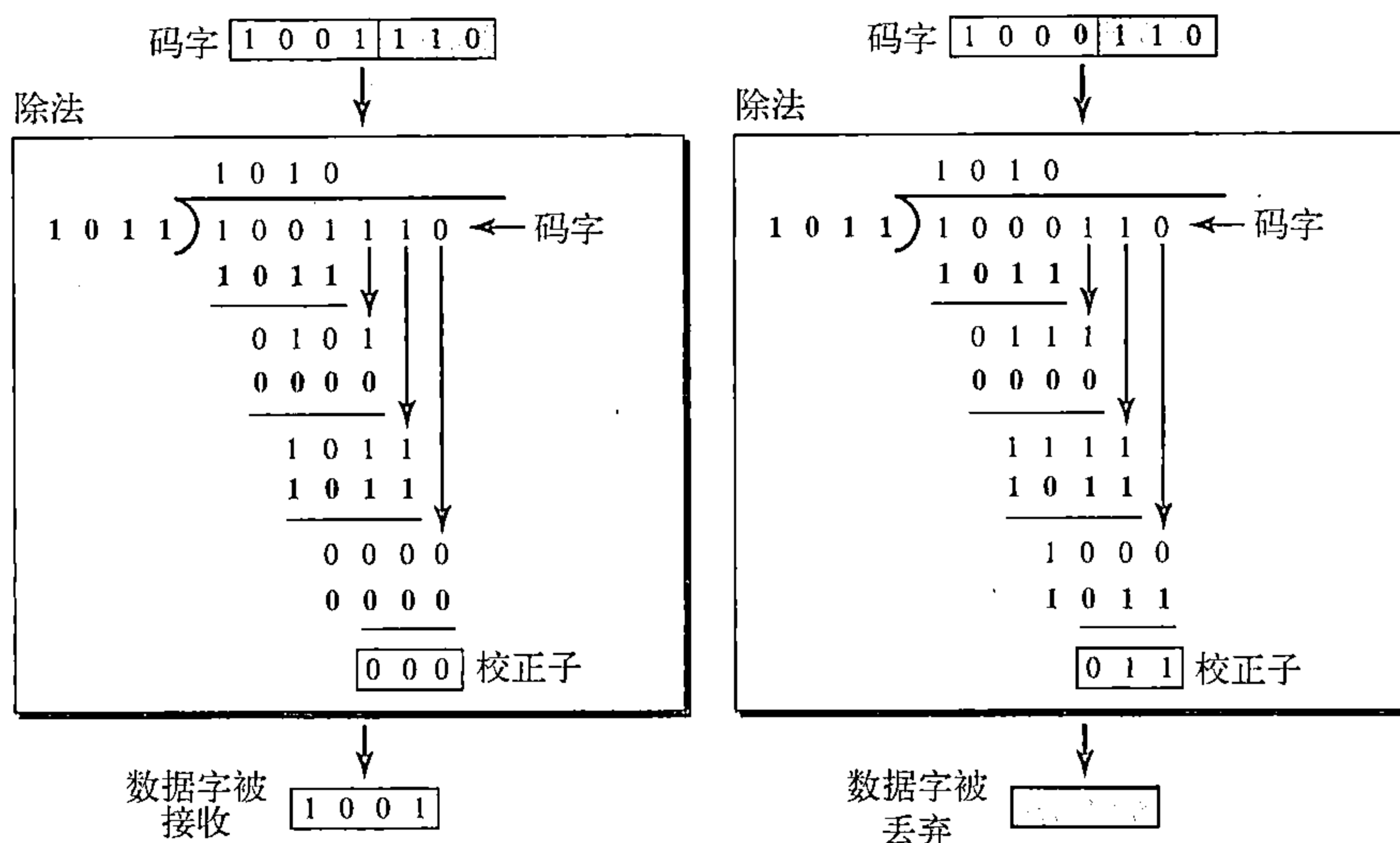


图10.16 两种情形下CRC译码器中的除法

### 除数

读者可能想知道如何选择除数1011。在本章后面我们会给出一些标准，但是通常它涉及抽象代数。

#### 10.4.2 硬件实现

循环编码的一个优点是使用少量的电子设备，编码器和译码器就可以用硬件轻易地并且便宜地实现。还有，硬件实现增加了校验位和校正子位计算的比率。在本节，我们尝试逐步说明这个过程。但是，本节是可选的，不会影响对本书其他部分的理解。

### 除数

让我们先来考虑除数。需要注意如下几点：

1. 除数重复地与除数部分进行异或运算；
2. 除数有 $n-k+1$ 位，或者是预定义的，或者全为0。换言之，这些位不会随数据字改变。在前面的例子中，除数或者是1011或者是0000。具体选哪一个取决于异或运算中起作用的增加的数据位（被除数）部分的最左边一位；

3. 进一步说明在异或运算中除数只需要 $n-k$ 位。不需要最左边的一位，因为无论这个位的值是什么运算，其结果永远是0。原因是异或操作的输入或者都是0或者都是1。在前面的例子中，只有3位（而不是4位）实际用于异或运算中。

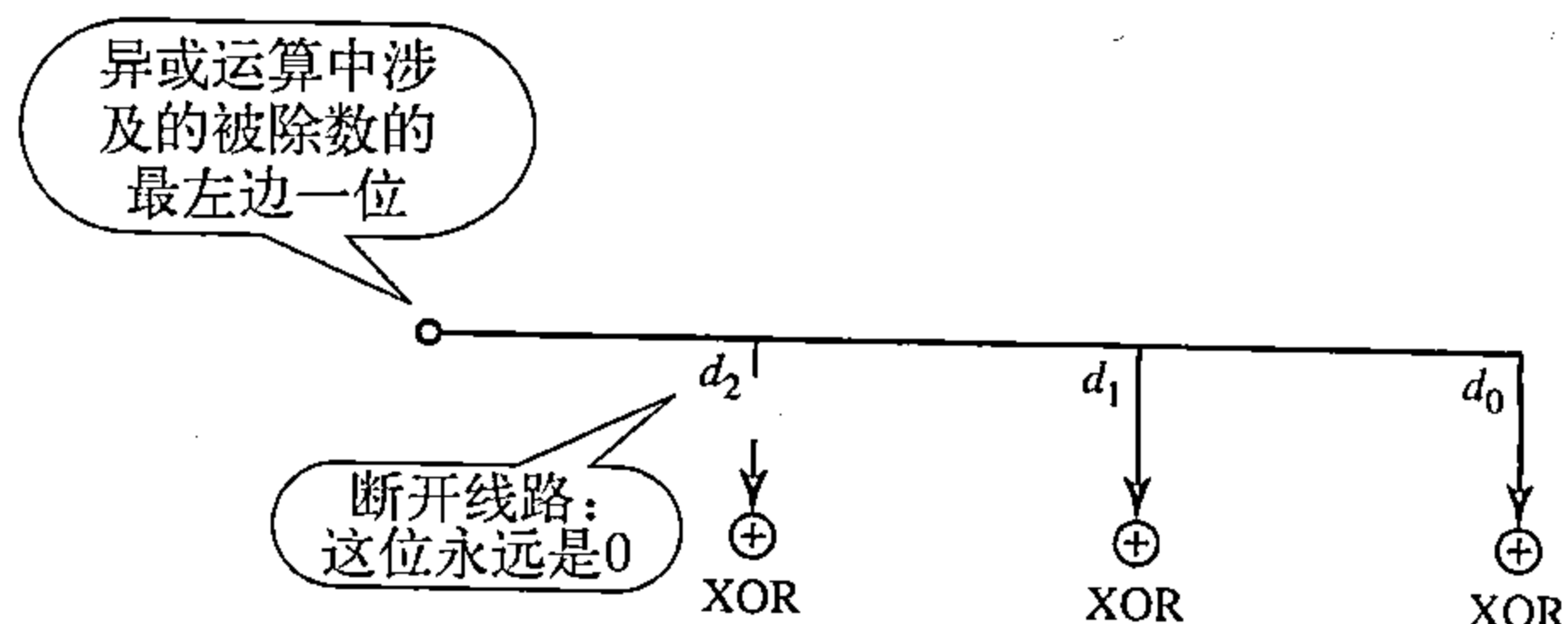


图10.17 CRC中除数的硬件设计

根据这几条，如果我们知道除数的模式，就能生成可用于循环编码的固定（硬件线路的）除数。图10.17给出了这样一个用于前面例子的设计，还显示了用于运算的异或设备。

**注意：**如果用于这一步的被除数部分的最左边一位是1，除数位（ $d_2d_1d_0$ ）是011，如果最



左边一位是0, 则除数位是000。这个设计根据最左边位做出正确的选择。

### 增加后的数据字

在图10.15用纸笔进行的除法运算过程, 我们表示增加后的数据字的位置不变, 而将除数向右位移, 每一步移1位。除数各位与增加后的数据字各位正好对齐。既然除数是固定的, 那么我们需要把增加后的数据字向左位移 (反方向), 使得除数的各位正好对齐。不需要保存增加后的数据字的各位。

### 余数

在前面的例子中, 余数的长度是3位 (一般 $n-k$ 位)。可以使用3个寄存器 (单个位存储设备) 来保存这些位。为了得到除法的最终余数, 需要修改除法过程。下面是以硬件 (或者甚至以软件) 模拟除法的过程。

1. 假设余数初始为全0 (我们的例子中是000)。
2. 在每个时间节拍 (增加后的数据字每到来1位), 重复以下两个处理过程:
  - a. 我们使用最左边的位来决定除数 (011或000)。
  - b. 余数的其他2位和增加后数据字的下一位 (一共3位) 与3位除数进行异或运算得出下一个余数。

图10.18表示了这个模拟过程。但是注意, 这不是最终设计, 最终设计将会有更大的改进。

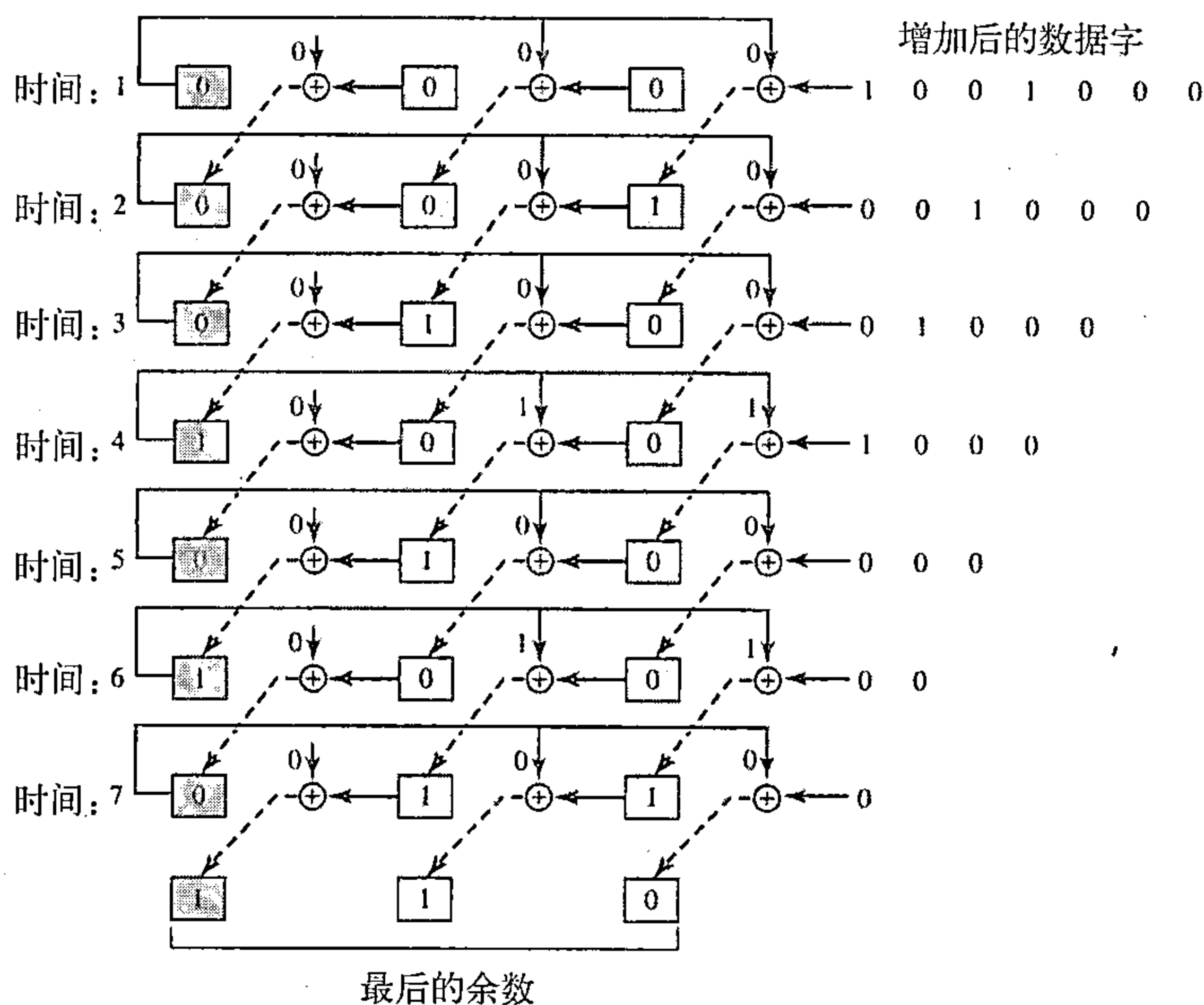


图10.18 CRC编码器中除法模拟

每个时间节拍用不同的时间表示, 增加后数据字的一位用于异或运算。如果仔细观察这个设计有7个步骤, 而用纸笔运算时只有4个步骤。增加了前3个步骤是为了使得每个步骤一样以及每个步骤的设计一样。步骤1、2和3把前3位压进余数寄存器; 步骤4、5、6和7与纸笔运算相匹配。注意步骤4到7的余数寄存器中的值正好与纸笔运算中的值相匹配。最后的余数也一样。

上面的设计只是为了演示, 它需要简化才能用于实际。首先, 不需要保持余数各位的中间值, 只需要最后的值。因此, 只需要3个寄存器而不是24个。在异或运算后, 不需要前一个



余数的各位值。还有，不需要21个异或设备，两个就足够了，因为如果一个位是0，异或运算的输出就是另一个位的值。另一个位就可以用于输出。使用这两个修改，设计变得极其简单也更便宜，如图10.19所示。

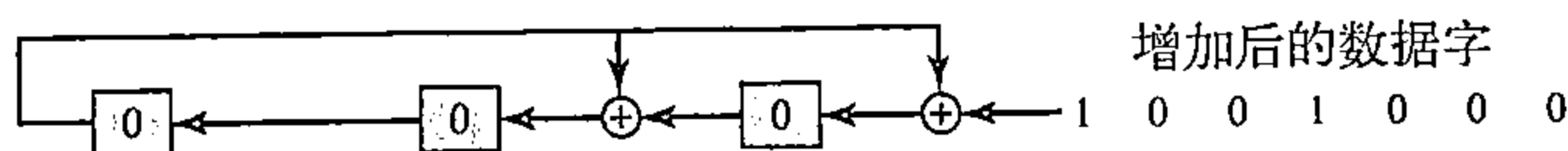


图10.19 使用移位寄存器的CRC编码器设计

但是，需要把寄存器改成移位寄存器。在一个时钟节拍期间，1位移位寄存器保存1位。在每个时间节拍点，移位寄存器从输入端口接收一个位，存储新的位并显示在输出端口。直到下一个输入到来其内容和输出保持一致。当把若干个1位移位寄存器连接在一起，它看起来好像寄存器的内容在移位。

#### 一般设计

一个编码器和译码器的一般设计如图10.20所示。

注意：在编码器和译码器中有 $n-k$ 个1位移位寄存器。最多有 $n-k$ 个异或设备，但是除数通常在它们的模式中有若干个0，这可以减少设备的数量。还要注意：作为输入是数据字，而不是增加后的数据字，因为数据字中的位输入到编码器后，附加的位（全为0）不会影响最右边的异或。当然，在生成校验位前，这个处理还需要继续另外 $n-k$ 个步骤。这个事实是对这个设计的批评之一。更好的方案设计能消除这个等待时间（校验位在 $k$ 步后就生成），但是我们把这个留作读者的研究课题。然而，在译码器中，整个码字在校正子生成前必须输入到译码器。

注意：

如果除数中相应位是0，  
那么就没有除数  
线路和异或

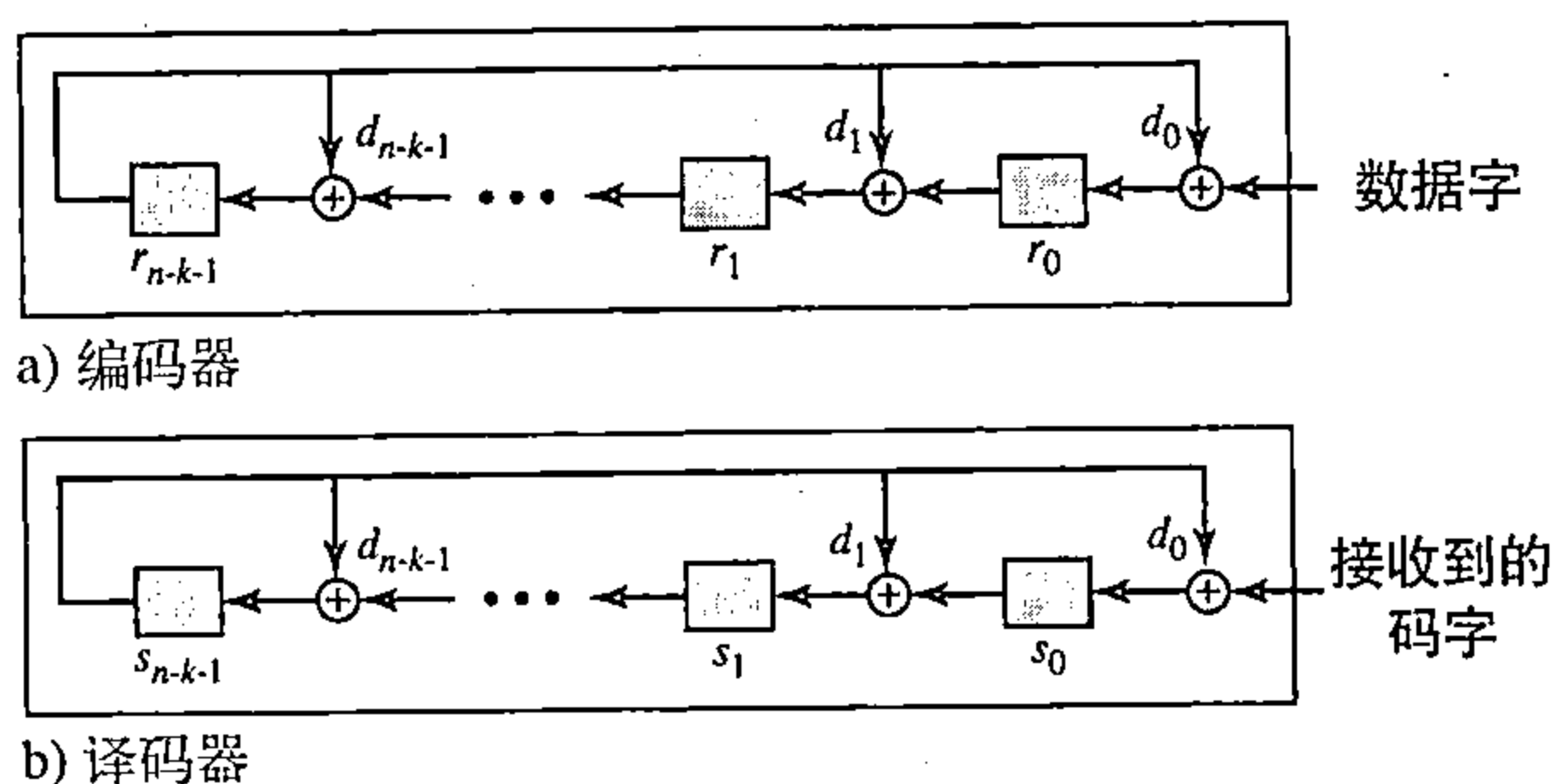


图10.20 CRC编码中编码器和译码器的一般设计

#### 10.4.3 多项式

理解循环编码以及如何分析它们的更好办法是将它们表示为多项式。本节也是可选的。

若干个0和1组成的模式可以表示为以0和1为系数的多项式（polynomial）。每一项的幂次表示位所在的位置，系数表示位的值。图10.21给出了一个二进制模式和它的多项式表示。在图10.21a中，说明了如何把二进制模式翻译成多项式；在图10.21b中说明了如何通过除去系数为0的项并用 $x$ 代替 $x^1$ 、1代替 $x^0$ 来简化多项式。

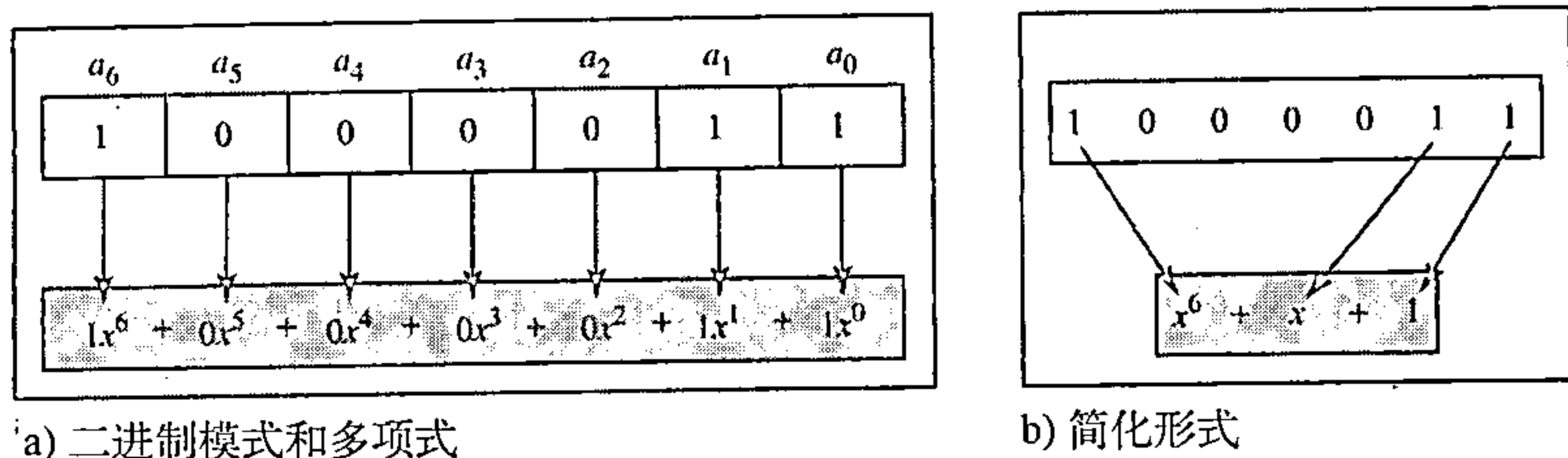


图10.21 用多项式表示二进制字

图10.21说明了一个显而易见的好处，7位模式可以用3项表示。当遇到诸如 $x^{23}+x^3+1$ 的多项式时，好处更加突出。这里模式是24位长（3个1和21个0），而用多项式表示时只有3项。



### 多项式的次数

多项式的次数是多项式中的最高幂次。例如，多项式 $x^6+x^3+1$ 的次数是6。注意多项式的次数比二进制模式中位数少1。这种情况下位模式有7位。

### 多项式加减法

数学上的多项式加减是通过加减相同幂次项的系数。此时系数只有0和1，加是模2加法。有两个结论。第一，加和减相同。第二，加减是删除相同项把不同项合在一起。例如 $x^5+x^4+x^2$ 和 $x^6+x^4+x^2$ 的结果是 $x^6+x^5$ 。 $x^4$ 和 $x^2$ 被删除。然而，注意：如果3个多项式相加，有3个项 $x^2$ ，就删除其中2个保留1个。

### 项乘法和除法

在这个算术中，一个项乘以另一个项很简单，只是幂次相加。例如 $x^3 \times x^4$ 是 $x^7$ 。对于除法，只需第一个幂次减去第二个幂次即可。例如 $x^5/x^2$ 是 $x^3$ 。

### 两个多项式相乘

一个多项式乘以另一个多项式通过项乘项完成。第一个多项式的每一个项必须乘以第二个多项式的所有项。然后相同项对被删除化，简其结果。一个实例如下：

$$(x^5+x^3+x^2+x)(x^2+x+1)=x^7+x^6+x^5+x^5+x^4+x^3+x^4+x^3+x^2+x^3+x^2+x=x^7+x^6+x^3+x$$

### 一个多项式除另一个多项式

从概念上来说，多项式除法与编码器中讨论的二进制除法相同。被除数位的第一项除以除数的第一项得到商的第一项。然后，将这一项乘以除数的结果从被除数中减去，重复这个过程直到被除数次数小于除数为止。我们会在本章稍后给出一个除法例子。

### 移位

二进制模式时常左移或右移若干位。左移表示在右边加上若干个额外0；右移表示删除右边若干位。左移通过每一项乘以 $x^m$ 实现，这里 $m$ 是移动的位数；右移通过每一项除以 $x^m$ 实现。下面说明了左移和右移。注意在多项式中没有负幂次。

左移3位：10011变成10011000  $x^4+x+1$ 变成 $x^7+x^4+x^3$

右移3位：10011变成10  $x^4+x+1$ 变成 $x$

在图10.15编码器中的增加后数据字实际上是左移了若干位。还要注意，连接两个位模式就是左移第一个多项式再加上第二个多项式。

### 使用多项式的循环编码编码器

已经讨论过了多项式运算，现在说明如何从数据字生成码字。图10.22是图10.15的多项式的表示形式。可以看到这个过程更短。数据字1001表示为 $x^3+1$ 。除数1011表示为 $x^3+x+1$ 。为了得到增加后的数据字，先把数据字左移3位（乘以 $x^3$ ）。其结果是 $x^6+x^3$ 。除法是直接的，用被除数的第一项 $x^6$ 先除以除数的第一项 $x^3$ 。商的第一项是 $x^6/x^3=x^3$ 。然后，被除数减去 $x^3$ 乘以除数（根据前面的减法定义）得到的结果。结果的第一项是 $x^4$ ，它的次数大于除数的次数。继续除直到余数的次数小于除数的次数。

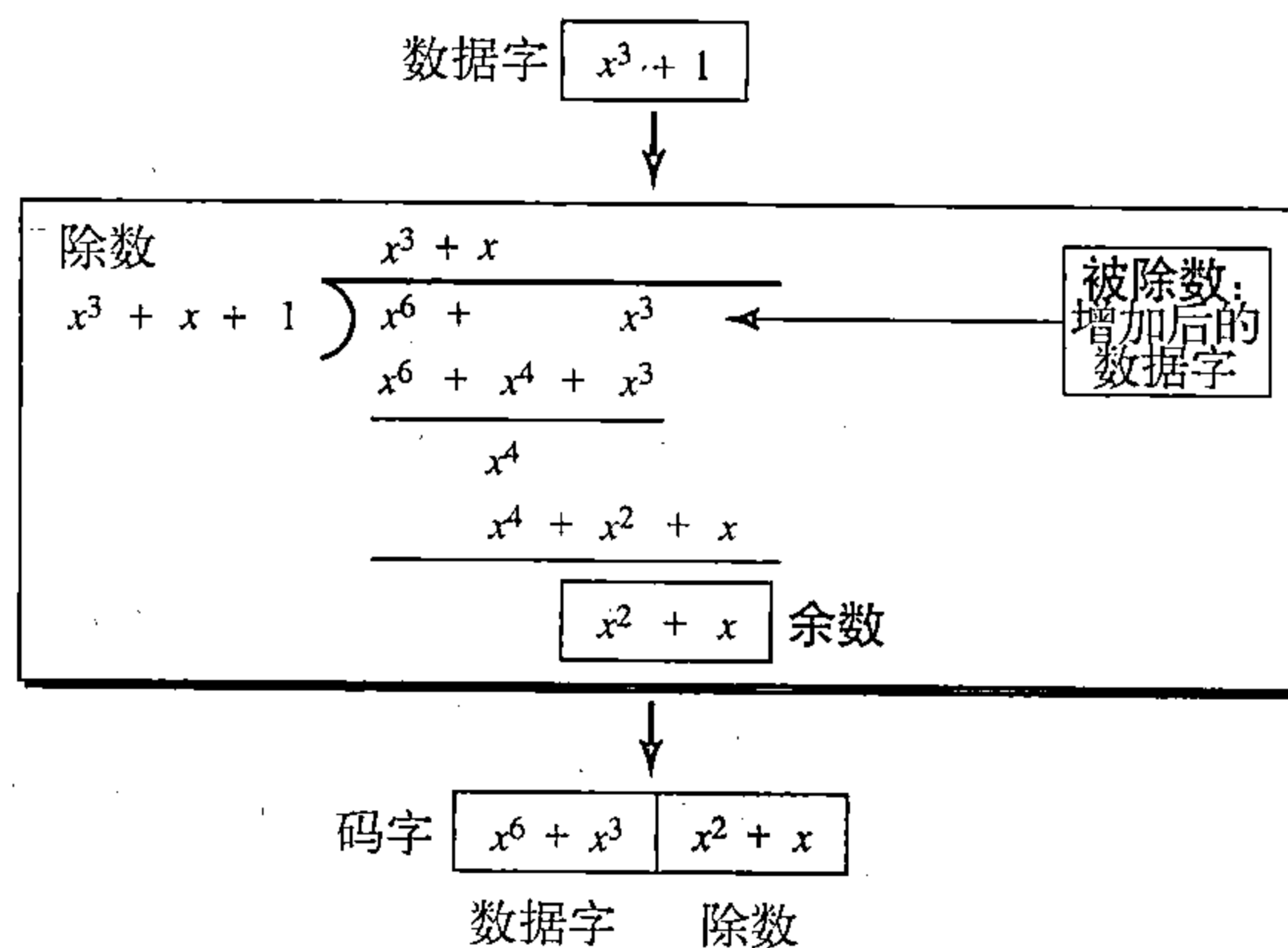


图10.22 使用二项式的CRC除法



可以看到在这个例子中，多项式表示简化了除法运算，因为这里不再需要涉及全0除数的两个步骤。（当然，人们可以将二进制除法中全0除数的步骤也去掉。）在多项式表示中，除数一般指生成多项式 $t(x)$ 。

循环编码中的除数通常称为生成多项式或简称为生成器。

#### 10.4.4 循环编码分析

我们可以使用多项式来分析循环编码的能力。我们定义了如下多项式，这里 $f(x)$ 是二进制系数的多项式。

数据字： $d(x)$     码字： $c(x)$     生成多项式： $g(x)$

校正子： $s(x)$     差错： $e(x)$

如果 $s(x)$ 不为0，那么一位或多位被破坏。但是，如果 $s(x)$ 为0，或者没有位被破坏或者译码器无法检测到任何差错。

在循环编码中，

1. 如果 $s(x) \neq 0$ ，一位或多为被破坏。
2. 如果 $s(x) = 0$ ，
  - a. 没有位被破坏，或者
  - b. 有一些位被破坏，但是译码器无法检测到。

在分析中，要找到生成多项式 $g(x)$ 使得能检测出那些想检测出的差错类型的标准。让我们先找到发送的码字、差错、接收到的码字和生成多项式之间的关系。可以说

$$\text{接收到的码字} = c(x) + e(x)$$

换言之，接收到的码字是发送的码字和差错之和。接收方用 $g(x)$ 除接收到的码字得到校正子。可以写成

$$\frac{\text{接收到的码字}}{g(x)} = \frac{c(x)}{g(x)} + \frac{e(x)}{g(x)}$$

等式右边的第一项没有余数（根据码字的定义）。因此，校正子实际上是右边第二项的余数。如果这一项没有余数（校正子=0），即 $e(x)$ 为0或者 $e(x)$ 可以被 $g(x)$ 整除。不必担心第一种情况（没有差错），第二种情况很重要。那些可以被 $g(x)$ 整除的差错无法被捕捉到。

在循环编码中，那些可以被 $g(x)$ 整除的差错无法被捕捉到。

让我们先说明一些特殊的差错，看看它们如何被设计好的 $g(x)$ 捕捉到。

##### 单个位差错

$g(x)$ 的组成应该怎样才能保证检测到单个位差错？单个位差错是 $e(x)=x^i$ ，这里 $i$ 是差错位的位置。如果单个位差错被捕捉到，那么 $x^i$ 不能被 $g(x)$ 整除。（注意：当我们说不能整除时，表示它有余数。）如果 $g(x)$ 至少有两项（通常是这样的）并且 $x^0$ 的系数不为0（最右边的位为1），那么 $e(x)$ 不能被 $g(x)$ 整除。

如果生成多项式至少有两项并且 $x^0$ 的系数为1，那么所有单个位差错都可以被捕捉到。

#### 例10.15

如下哪个 $g(x)$ 可以保证捕捉到单个位差错？对于每种情况，不能捕捉的差错是什么？

- a.  $x+1$



b.  $x^3$ 

c. 1

解:

a. 没有一个 $x^i$ 可以被 $x+1$ 整除。换言之,  $x^i/(x+1)$  总有余数。因此, 校正子不是零。任何单个位差错都可以被捕捉到。

b. 如果 $i$ 大于或等于3,  $x^i$ 可以被 $g(x)$ 整除。 $x^i/x^3$ 的余数是0, 接收方便会被蒙蔽而认为没有差错, 虽然实际是一个差错。注意: 这种情况下, 被破坏的位必须位于第4位或更高位。能捕捉到位1到位3的单个差错。

c.  $i$ 的所有值使得 $x^i$ 可以被 $g(x)$ 整除。没有单个位差错可以被捕捉到。另外,  $g(x)$ 是无用的, 因为它表示的只是增加了 $n-k$ 个0后的数据字的码字。

### 两个独立的单个位差错

现在想像有两个独立的单个位差错。什么情况下这种类型的差错可以被捕捉到? 可以把这种差错表示为 $x^j+x^i$ 。 $i$ 和 $j$ 的值只表示差错的位置, 差 $j-i$ 表示两个差错的距离, 如图10.23所示。

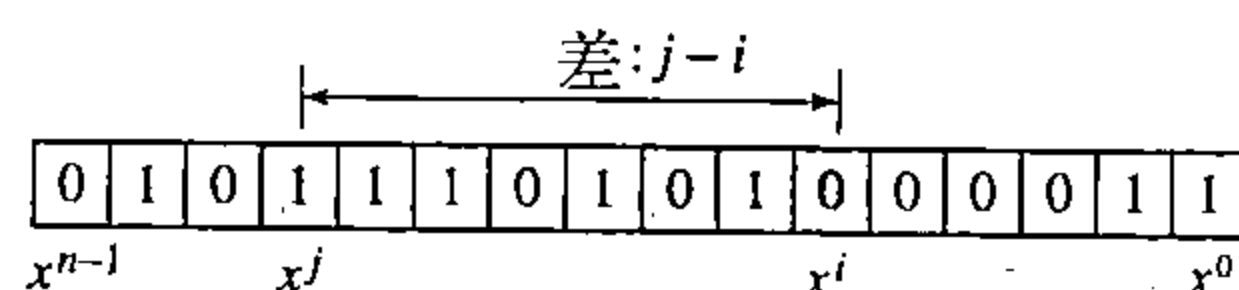


图10.23 使用多项式表示的两个独立的单个位差错

可以写成 $e(x)=x^i(x^{j-i}+1)$ 。如果 $g(x)$ 多于一项并且其中一项为 $x^0$ , 那么正如在前面看到的, 它不能整除 $x^i$ 。因此, 如果 $g(x)$ 能整除 $e(x)$ , 那么它一定能整除 $x^{j-i}+1$ 。换言之,  $g(x)$ 一定不能整除 $x^t+1$ , 这里 $t$ 在0和 $n-1$ 之间。但是 $t=0$ 是无意义的, 并且稍后会看到需要 $t=1$ 。这就表示 $t$ 应该在2和 $n-1$ 之间。

如果生成多项式不能整除 $x^t+1$  ( $t$ 在0和 $n-1$ 之间), 那么所有独立的双差错都能被检测到。

### 例10.16

求出以下与两个独立的单个位差错相关的生成多项式的情况。

a.  $x+1$ b.  $x^4+1$ c.  $x^7+x^6+1$ d.  $x^{15}+x^{14}+1$ 

解:

a. 对生成多项式来说, 这是一个很差的选择, 任何两个相邻的差错都不能被检测到。

b. 这个生成多项式不能检测到相隔4个位置的两个差错。这两个差错能位于任何位置, 但是如果它们的距离是4仍然无法被检测到。

c. 这是个好的选择。

d. 如果 $t$ 小于32 768, 这个多项式不能整除类型为 $x^t+1$ 的任何差错。这表示一个码字中两个独立的差错相邻或者最多离开32 768位都能被这个生成多项式检测到。

### 奇数个差错

有因子 $x+1$ 的生成多项式可以捕捉所有奇数个差错。这表示需要把 $x+1$ 作为任何生成多项式的因子。注意: 不是说生成多项式本身应该是 $x+1$ , 而是说它应该有因子 $x+1$ 。如果它只是 $x+1$ , 那么它不能捕捉到两个相邻的独立差错 (见前一节)。例如 $x^4+x^2+1$ 能捕捉到所有奇数个差错, 因为它可写作两个多项式 $x+1$ 和 $x^3+x^2+1$ 的乘积。



包含因子 $x+1$ 的生成多项式能检测到所有奇数个差错。

### 突发性差错

现在把分析扩展到突发性差错，它是所有差错中最重要的。一个突发性差错的形式是 $e(x) = (x^j + \dots + x^i)$ 。注意：突发性差错和两个独立的单个位差错的不同。第一种有两项或更多项；而第二种只有两项。我们可以分解出因子 $x^i$ 把差错 $e(x)$ 写成 $x^i(x^{j-i} + \dots + 1)$ 。如果生成多项式能检测到单个差错（对于生成多项式的最小条件），那么它不能整除 $x^i$ 。我们应该关注的是能整除 $x^{j-i} + \dots + 1$ 的那些生成多项式。换言之， $(x^{j-i} + \dots + 1)/(x^r + \dots + 1)$ 的余数一定不是0。注意分母是生成多项式。可能有三种情况：

1. 如果 $j-i < r$ ，余数永远不会是0。我们可以写成 $j-i = L-1$ ，这里 $L$ 是差错的长度。因此 $L-1 < r$ 、 $L < r+1$ 或 $L \leq r$ 。这表示所有长度小于或等于校验位数 $r$ 的突发性差错都会被检测到。

2. 在一些少见的情况中，如果 $j-i = r$ 或 $L = r+1$ ，校正子是0，差错无法被检测到。这可以在这些情况中得到验证，长度是 $r+1$ 的突发性差错无法被检测到的概率是 $(1/2)^{r-1}$ 。例如，如果我们的生成多项式是 $x^{14} + x^3 + 1$ ，这里 $r$ 是14，那么长度 $L=15$ 的突发性差错概率是 $(1/2)^{14-1}$ ，即几乎10 000中有1个无法被检测到。

3. 在一些少见的情况中，如果 $j-i < r$ 或 $L > r+1$ ，校正子是0，差错无法被检测到。这可以在这些情况中得到验证，长度大于 $r+1$ 的突发性差错无法被检测到的概率是 $(1/2)^r$ 。例如，如果我们的生成多项式是 $x^{14} + x^3 + 1$ ，这里 $r$ 是14，那么长度 $L=15$ 的突发性差错概率是 $(1/2)^{14}$ ，即几乎16 000中有1个无法被检测到。

- 所有 $L \leq r$ 的突发性差错都会被检测到。
- 所有 $L = r+1$ 的突发性差错有 $1-(1/2)^{r-1}$ 的概率会被检测到。
- 所有 $L > r+1$ 的突发性差错有 $1-(1/2)^r$ 的概率会被检测到。

### 例10.17

分析出下面与不同长度突发性差错相关的生成多项式的适用性。

- a.  $x^6 + 1$
- b.  $x^{18} + x^7 + x + 1$
- c.  $x^{32} + x^{23} + x^7 + 1$

解：

a. 这个生成多项式可以检测出所有长度小于或等于6位的突发性差错；长度为7的突发性差错无法被检测到概率是3%；长度大于或等于8的突发性差错无法被检测到概率是16%。

b. 这个生成多项式可以检测出所有长度小于或等于18位的突发性差错；长度为19的突发性差错无法被检测到概率是百万分之8；长度大于或等于20的突发性差错无法被检测到概率是百万分之4。

c. 这个生成多项式可以检测出所有长度小于或等于32位的突发性差错；长度为33的突发性差错无法被检测到概率是千万分之5；长度大于或等于34突发性差错无法被检测到概率是千万分之3。

### 总结

可以总结一下令人满意的生成多项式的标准：

高性能生成多项式需要有以下特性：

1. 它应该至少有两项。
2.  $x^0$ 项的系数应该是1。
3. 它应该不能整除 $x^t + 1$  ( $t$ 是2到 $n-1$ 之间)。
4. 它应该有因子 $x+1$ 。



## 标准多项式

常用协议中为CRC产生的标准多项式如表10.7所示。

表10.7 标准生成多项式

| 名 称    | 多 项 式                                                                             | 应 用     |
|--------|-----------------------------------------------------------------------------------|---------|
| CRC-8  | $x^8+x^2+x+1$                                                                     | ATM头部   |
| CRC-10 | $x^{10}+x^9+x^5+x^4+x^2+1$                                                        | ATM AAL |
| CRC-16 | $x^{16}+x^{12}+x^5+1$                                                             | HDLC    |
| CRC-32 | $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$ | LANs    |

### 10.4.5 循环编码的优点

我们已经看到循环编码在检测单个位差错、双差错、奇数个差错和突发性差错中有非常令人满意的性能。它们可以很容易地以软硬件实现。当以硬件实现时，速度尤其快，这使得循环编码成为许多网络的好选择。

### 10.4.6 其他循环编码

本节中讨论的循环编码很简单，校验位和校正子可以通过简单的代数计算得出。但是还有基于抽象代数的Galois域上更强功能的多项式，它们超出了本书的范围。这些编码中最令人感兴趣的是目前用于检错和纠错的Reed-Solomon编码（Reed-Solomon code）。

## 10.5 校验和

这里讨论的最后一个检错方法称为校验和（checksum）。校验和在因特网中被许多不属于数据链路层的协议使用。但是，在这里简单地讨论它以使关于检错的讨论更加完整。像线性和循环编码一样，校验和也是基于冗余的概念。将会在下后面的章节中看到，许多协议仍然使用校验和进行检错，虽然趋势是它将被CRC替代。这表示CRC还用于除了数据链路层之外的其他层。

### 10.5.1 概念

校验和的概念并不难。让我们用几个例子来解释它。

#### 例10.18

假设我们要发送到目的地的数据是5个4位数字。除了发送这些数字外，我们还发送这些数字的和。例如，如果这组数字是（7、11、12、0、6），那么我们发送（7、11、12、0、6、36），这里36是原来数字的和。接收方将这5个数字相加并将相加的结果与这和比较。如果两者相同，接收方就认为没差错，接收这5个数字并丢弃和。否则，就认为有差错，不接收这个数据。

#### 例10.19

如果我们发送和的负值（补数）——称为校验和，就会使接收方的工作更容易。在这个例子中，我们发送（7、11、12、0、6、-36）。接收方将所有接收到的数字（包括校验和）相加。如果结果是0，就认为没差错。否则就认为有差错。

### 10.5.2 反码

前面的例子中有一个主要的缺点。除了校验和以外，所有的数据都写做4位字（它们小于15）。一个解决办法是使用反码（one's complement）算法。在这个算法中，只使用 $n$ 位表示0



到 $2^n-1$ 的无符号数字<sup>①</sup>。如果这个数字多于 $n$ 位，那么最左边的额外位要加到最右边的 $n$ 位。在反码算法中，一个数的负数可用该数所有位取反（0变成1，1变成0）表示。这与从 $2^n-1$ 减去这个数字一样。

#### 例10.20

在反码运算中如何只用4位表示数字21？

解：

数字21的二进制表示是10101（需要5位）。可以把最左边的位加到最右边的4位上。  
 $0101+1=0110$ 或6。

#### 例10.21

在反码运算中如何只使用4位表示数字-6？

解：

在反码算法中，一个数的负数是将所有位取反。正6是0110，负6是1001。如果只考虑无符号数字，它是9。换言之，6的补数是9。在反码运算中，另一种求一个数的反码的方法是 $2^n-1$ 减去这个数（这里是16-1）。

#### 例10.22

用反码运算重做练习10.19。图10.24给出了发送方和接收方的处理过程。发送方先把校验和初始化为0，并将所有数据项与校验和（校验和被看做是一个数据项并用颜色显示）相加。其和是36。但是，36不能用4位表示。多出的2位与和相加得到约束的校验和是6。在图中，以二进制表示其详细过程。然后对和求反，结果是校验和为9（15-6）。现在，发送方发送6个数据项（包括校验和9）给接收方。接收方执行与发送方相同的处理。它先把所有数据项相加（包括校验和），结果是45。然后和变成15。对约束和求反变成0。因为校验和的值是0，这表示数据没有被破坏。接收方去掉校验和保存其他数据项。如果校验和不是0，整个分组就被丢弃。

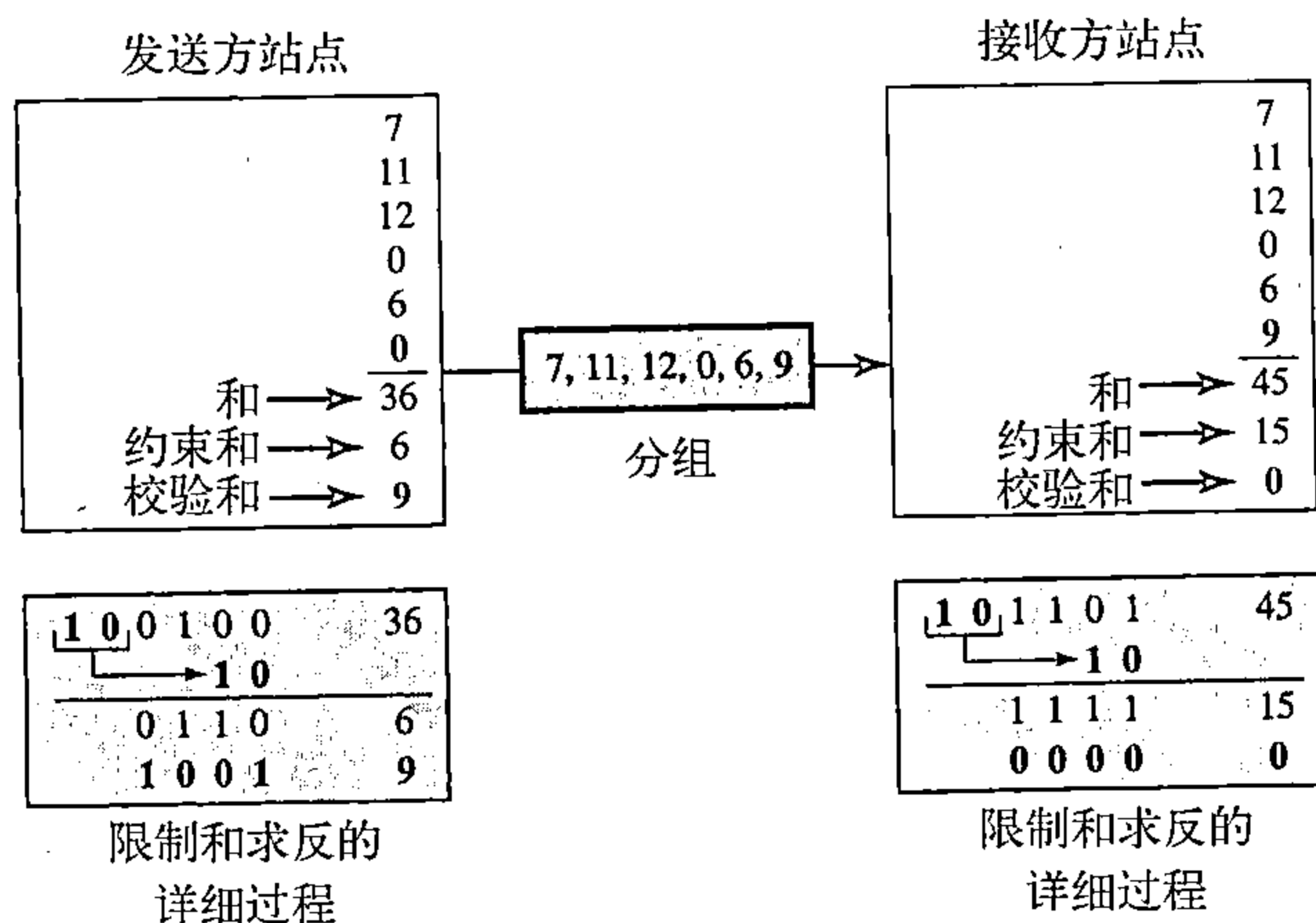


图 10.24

### 10.5.3 因特网校验和

传统上，因特网使用16位校验和。发送方通过下面的步骤计算校验和。

<sup>①</sup> 虽然，一个数的反码可以同时代表正数和负数，但这里我们只关心无符号数。



发送方站点:

1. 报文被划分为16位字。
2. 校验和字的值设为0。
3. 所有字包括校验和使用反码运算相加。
4. 对这个和求反变成校验和。
5. 校验和随数据一起发送。

接收方使用如下步骤用于差错检测。

接收方站点:

1. 报文 (包括校验和) 被划分成16位字。
2. 用反码加法将所有字相加。
3. 对该和求反生成新的校验和。
4. 如果校验和的值是0, 接收报文; 否则就丢弃报文。

校验和的特性 (把字看做是数字并相加后求反) 完全适用于软件实现。可以写小程序在接收方站点计算校验和或检查报文的有效性。

### 例10.23

让我们计算8个字符 (Forouzan) 文本的校验和。该文本需要划分成2个字节 (16位) 的字。使用ASCII (见附录A) 把每个字符变成2位十六进制数。例如, F表示成0x46, o表示成0x6F。图10.25说明了如何在发送方站点和接收方站点计算校验和。在图10.25a中, 第一列部分的和是0x36。我们保持最右边的数字 (6) 并将最左边的数字 (3) 作为进位插入到第二列中。每一列重复这个过程。十六进制数见附录B。

注意: 如果受到任何破坏, 接收方重新计算的校验和将不全是0。把这个留到练习中。

|   |   |   |   |           |
|---|---|---|---|-----------|
| 1 | 0 | 1 | 3 | 进位        |
| 4 | 6 | 6 | F | (Fo)      |
| 7 | 2 | 6 | F | (ro)      |
| 7 | 5 | 7 | A | (uz)      |
| 6 | 1 | 6 | E | (an)      |
| 0 | 0 | 0 | 0 | 校验和 (初始值) |
| 8 | F | C | 6 | 和 (部分和)   |
| 3 |   |   |   | 进位        |
| 8 | F | C | 7 | 和         |
| 7 | 0 | 3 | 8 | 校验和 (发送)  |

a) 发送站点校验和

|   |   |   |   |           |
|---|---|---|---|-----------|
| 1 | 0 | 1 | 3 | 进位        |
| 4 | 6 | 6 | F | (Fo)      |
| 7 | 2 | 6 | F | (ro)      |
| 7 | 5 | 7 | A | (uz)      |
| 6 | 1 | 6 | E | (an)      |
| 7 | 0 | 3 | 8 | 校验和 (接收到) |
| F | F | F | E | 和 (部分和)   |
| 1 |   |   |   | 进位        |
| F | F | F | F | 和         |
| 0 | 0 | 0 | 0 | 校验和 (新)   |

b) 接收站点校验和

图 10.25

### 性能

传统的校验和使用较少的 (16) 位来检测任何长度 (有时是数千位) 报文中的差错。但是, 在差错检测能力上它没有CRC强。例如, 增加一个字的值而另一个字的以相同值减小, 因为和以及校验和不变, 所以这两个差错无法被检测到。还有, 如果多个字的值增加, 但是总的变化量是65535的倍数, 则和以及校验和也不变, 这表示差错也无法被检测到。Fletcher和Adler建议使用一些带权重的校验和, 每个字乘以一个与它在文本中位置相关的一个数 (称为它的权重)。这就消除了提到的第一个问题。但是因特网的趋势, 尤其在新协议的设计中, 是由CRC代替校验和。



## 推荐读物

为了深入讨论与本章有关的主题，我们推荐下列读物。在本书末列出括号[...]中的参考资料。

## 读物

许多优秀的书籍致力于差错编码。在这些书籍中我们推荐[Ham80]、[Zar02]、[Ror96]和[SWE04]。

## 请求评论

因特网中校验和应用的讨论可以在RFC1141中找到。

## 本章小结

- 传输期间数据可能被损坏，某些应用要求检错与纠错。
- 单个位差错是数据单元中仅有一位改变，突发性差错是数据单元中有两个或多个位改变。
- 为了检错和纠错，须与数据一起发送附加（冗余）位。
- 有两种纠错方法：向前纠错与重发纠错。
- 可将编码方案分为两大类：块编码与卷积编码。
- 在编码中，须采用模-2运算，这个运算非常简单；加法与减法是相同的。对加法与减法可用XOR（异或）运算。
- 在块编码中，将报文划分为块，每块 $k$ 位，称为数据字。每块增加 $r$ 个冗余位，使长度为 $n=k+r$ ，形成 $n$ 位的块称为码字。
- 在块编码中，用下列两个条件进行检错：
  - a. 接收方有（或可找到）一张有效码字表；
  - b. 原来的码字变成无效码字。
- 两个字的汉明距离是对应位不同的个数，一组字的最小汉明距离是该组中所有字对的最小汉明距离。
- 在任何情况下，为了保证能检测到 $s$ 个差错，在块编码中的最小汉明距离必须是 $d_{\min}=s+1$ 。在任何情况下，为了保证能纠正 $t$ 个差错，在块编码中的最小汉明距离必须是 $d_{\min}=2t+1$ 。
- 在线性块编码中，任何两个有效代码的异或（XOR）生成另一个有效代码。
- 在简单奇偶校验编码中 $n=k+1$ 与 $d_{\min}=2$ ，它能检测代码的单个位差错。
- 本书讨论的汉明编码都是 $d_{\min}=3$ ，在这些编码中 $m$ 与 $n$ 之间的关系是 $n=2m-1$ 。
- 循环编码是具有附加性质的特殊的线性块编码。在循环编码中，一个码字循环移位必是另一码字。
- 一类称为循环冗余校验（CRC）的循环编码用于局域网（LAN）和广域网（WAN）。
- 0和1的位组可表示系数为0和1的代数多项式。
- 传统上因特网一直使用16-位校验和，并使用反码运算。这种运算仅用 $n$ 位表示0到 $2^n-1$ 之间的无符号数。

## 习题

### 复习题

1. 单个位差错与突发性差错有什么不同？
2. 叙述检错与纠错中冗余的概念。



3. 区别向前纠错与重发纠错。
4. 线性块编码的定义是什么？循环编码的定义是什么？
5. 什么是汉明距离？什么是最小汉明距离？
6. 简单的奇偶校验与两维奇偶校验有什么关系？
7. 在CRC中，表示下列实体之间关系（长度是指位的个数）：
  - a. 数据字长度与码字长度
  - b. 除数长度与余数长度
  - c. 生成多项式的次数与除数长度
  - d. 生成多项式的次数与余数长度
8. 在校验和计算中，数据项相加采用哪一种算术运算？
9. 什么类型的差错用校验和不能检测出来？
10. 校验和的值是否可为全0（二进制）？说出理由。校验和的值是否可为全1（二进制）？说出理由。

### 练习题

11. 一个2ms的突发性噪声在下列数据传输速率时造成的最大影响是什么？
  - a. 1500bps
  - b. 12kbps
  - c. 100kbps
  - d. 100Mbps
12. 进行异或运算（符号 $\oplus$ 表示异或）：
  - a.  $(10001) \oplus (10000)$
  - b.  $(10001) \oplus (10001)$ （由结果推断出什么？）
  - c.  $(11100) \oplus (00000)$ （由结果推断出什么？）
  - d.  $(10011) \oplus (11111)$ （由结果推断出什么？）
13. 按表10.1，发送方发送数据字10。一个3位突发性差错损坏了码字，接收方能否检测出差错？说出理由。
14. 按表10.2，发送方发送数据字10。如果一个3位突发性差错损坏了码字的前3位，接收方能否检测出差错？说出理由。
15. 下列代码字的汉明距离是多少？
  - a.  $d(10000, 00000)$
  - b.  $d(10101, 10000)$
  - c.  $d(11111, 11111)$
  - d.  $d(000, 000)$
16. 对下列情况，求最小汉明距离：
  - a. 检测2个差错
  - b. 纠正2个差错
  - c. 检测3个差错与纠正2个差错
  - d. 检测6个差错与纠正2个差错
17. 按表10.2，如果接收到下列码字，试问数据字是什么？
  - a. 01011
  - b. 11111
  - c. 00000
  - d. 11011
18. 证明表10.8所示的编码不是线性编码，只须找到一种情形违反线性的性质。
19. 虽然数学上能证明简单的奇偶校验编码是线性编码，对表10.3中5对代码用手工方法检测其线性性。
20. 说明表10.4中的汉明编码 $C(7, 4)$ ，通过检测下列各情况的编码可检测出2位差错但不一定能检测出3位差错。在突发性差错中，字符“ $\nabla$ ”是指没有差错，字符“E”是指有差错。
  - a. 数据字：0100 突发性差错： $\nabla E E \nabla \nabla \nabla \nabla$

表10.8 练习10.18

| 数 据 字 | 码 字   |
|-------|-------|
| 00    | 00000 |
| 01    | 01011 |
| 10    | 10111 |
| 11    | 11111 |



- b. 数据字: 0111 突发性差错: E ▽ ▽ ▽ ▽ ▽ E  
 c. 数据字: 1111 突发性差错: E ▽ E ▽ ▽ ▽ E  
 d. 数据字: 0000 突发性差错: E E E ▽ ▽ ▽ ▽
21. 说明表10.4中的汉明编码 $C(7, 4)$ , 在下列各情况通过检验代码可检测出1位差错但不一定能检测出多位差错。在突发性差错中, 字符“▽”是指没有差错, 字符“E”是指有差错。
- a. 数据字: 0100 突发性差错: E ▽ ▽ ▽ ▽ ▽ ▽  
 b. 数据字: 0111 突发性差错: ▽ E ▽ ▽ ▽ ▽ ▽  
 c. 数据字: 1111 突发性差错: E ▽ ▽ ▽ ▽ ▽ E  
 d. 数据字: 0000 突发性差错: E E ▽ ▽ ▽ ▽ E
22. 虽然可证明表10.6中代码既是线性的又是循环的, 仅用两次测试证明部分事实:  
 a. 测试码字0101100具有循环性质;  
 b. 测试码字0010110和1111111具有线性性质。
23. 我们需要至少11位的数据字, 求具有 $d_{\min}=3$ 的汉明编码 $C(n, k)$ 的 $n$ 与 $k$ 值。
24. 对相应的多项式进行下列运算:  
 a.  $(x^3+x^2+x+1)+(x^4+x^2+x+1)$   
 b.  $(x^3+x^2+x+1)-(x^4+x^2+x+1)$   
 c.  $(x^3+x^2) \times (x^4+x^2+x+1)$   
 d.  $(x^3+x^2+x+1)/(x^2+1)$
25. 试回答下列问题:  
 a. 101110代表的多项式是什么?  
 b. 101110左移3位其结果是什么?  
 c. 用多项式重做b。  
 d. 101110右移4位其结果是什么?  
 e. 用多项式重做d。
26. 下列哪个生成多项式能检测出单个位差错?  
 a.  $x^3+x+1$   
 b.  $x^4+x^2$   
 c. 1  
 d.  $x^2+1$
27. 用表10.7提到的CRC-8多项式, 回答下列问题:  
 a. 它能检测出单个位差错吗? 说出其理由。  
 b. 它能检测出长度为6的突发性差错吗? 说出其理由。  
 c. 检测出长度为9的突发性差错的概率是多少?  
 d. 检测出长度为15的突发性差错的概率是多少?
28. 用表10.7提到的CRC-32多项式, 回答下列问题:  
 a. 它能检测出单个位差错吗? 说出其理由。  
 b. 它能检测出长度为16的突发性差错吗? 说出其理由。  
 c. 检测出长度为33的突发性差错的概率是多少?  
 d. 检测出长度为55的突发性差错的概率是多少?
29. 假定为偶校验, 求下列数据单元的校验位:  
 a. 1001011  
 b. 0001100  
 c. 1000000  
 d. 1110111
30. 已知数据字1010011110和除数10111,  
 a. 指出发送站点产生的码字(用二进制除法)。  
 b. 指出接收站点校验的码字(无差错)。
31. 用多项式重做练习30。



32. 发送方须发送4个数据项：0x3456、0xABCC、0x02BC和0xEEEE，回答下列问题：
- a. 求发送站点的校验和；
  - b. 如果不存在差错，求接收站点的校验和；
  - c. 如果第二个数据项变成0xABCE，求接收站点的校验和；
  - d. 如果第二个数据项变成0xABCE和第三个数据项变成0x02BA，求接收站点的校验和。
33. 指出用校验和处理一个特殊情况的问题。发送方有两个数据项0x4567和0Xba98要发送。试问校验和的值是什么？



## 第11章 数据链路控制

数据链路层的两个主要功能是数据链路控制和介质访问控制。前者，即数据链路控制的功能是用来处理两个邻近节点之间通信，即节点到节点通信的设计和程序，我们将在此章节中阐述这一功能。而后者，数据链路层的第二个功能是介质访问控制，或者说如何共享链路，将在第12章中阐述。

**数据链路控制**（data link control）的功能包括成帧、流量控制和差错控制，以及能提供帧在节点之间流畅且可靠传输的软件实现协议。在这一章，首先阐述成帧，或者说如何组织物理层所承载的位，然后再阐述流量控制和差错控制。关于差错的检测和纠正的技术已在第10章阐述过了。

我们需要协议来实现数据链路控制。每一个协议都是一组以软件实施的规则，并由在数据链路层进行数据交换涉及的两个节点来运行。我们将阐述五个协议：两个用于无噪声（理想的）通道，三个用于有噪声（现实的）通道。第一类的协议并不能真正地实施，而是为理解第二类协议提供了基础。

在阐述了这五个协议后，我们将用一个实例，说明面向位的协议如何通过使用高级数据链路控制协议（HDLC）来实现。并阐述一个流行的面向字节的协议，即点对点协议（PPP）。

### 11.1 成帧

位于物理层的数据传递是指以信号的形式从源端传输到目的端。物理层提供位同步以保证发送方和接收方能使用相同的位周期和时序。

另一方面，数据链路层需将位组合成帧，并使帧与帧之间是可识别的。邮局系统就实践了一种成帧形式。将一份信插入一个信封的简单动作就将一段信息与另一段信息分离开来；信封就是一种分隔符。另外，既然邮局系统是一种多对多的传输设施，所以每个信封都标明了发送方和接收方的地址。

在数据链路层中，通过添加发送方地址和接收方地址，**成帧**（framing）将一条从源端到目的端的报文分离开来，或者将到不同目的端的报文分离开来。目的地址定义了分组要去的地方，而发送方地址帮助接收方确认接收。

尽管整条报文都应该分组为一个帧，但通常却不这么做。一个原因是一个帧会很大，将使得流量和差错控制变得十分低效。当一个巨大的帧承载一条报文时，即使一个字节的错误也需要重新传递整条报文。

#### 11.1.1 固定长度成帧

帧的大小可以是固定的，也可以是可变的。**固定大小成帧**（fixed-size framing），不需要定义帧的边界，长度本身可以用作分隔符。这种成帧类型的一个例子是ATM广域网，它用称为信元的固定长度帧。我们将在第18章讨论ATM。

#### 11.1.2 可变长度成帧

此章中我们主要讨论在局域网中盛行的**可变长度成帧**（variable-size framing）。在可变长度成帧中，需要一种方法来规定一个帧的结束和下一个帧的开始。过去，主要使用两种方法：



面向字符的方式和面向位的方式。

### 面向字符协议

在面向字符协议 (character-oriented protocol) 中, 数据用来自诸如ASCII (参见附录A) 编码系统的8位字符传输。头部通常携带源端地址和目的端地址以及其他控制信息, 而尾部携带检错或纠错冗余位, 也是8位的倍数。为了将一个帧与下一个分离开来, 在帧的开始和结束增加了8位 (1字节) 的标记 (flag)。标记由与协议相关的特殊字符组合而成, 标明一个帧的开始和结束。图11.1说明了在面向字符协议中帧的格式。

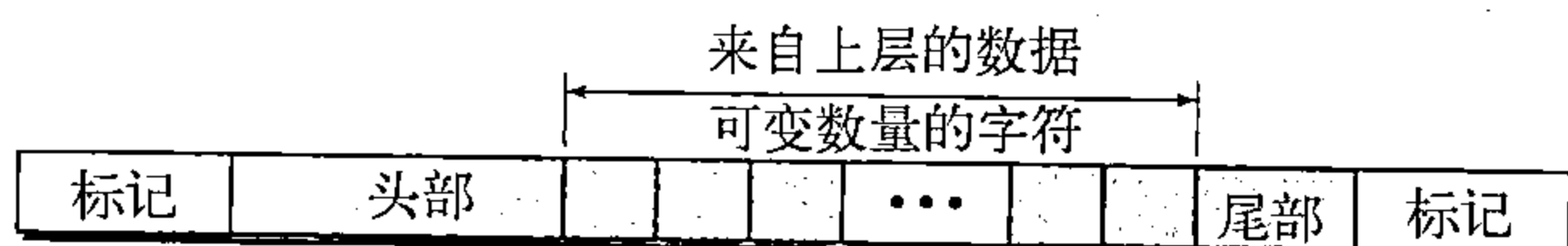


图11.1 面向字符协议中的帧

当数据链路层只交换文本时, 流行面向字符成帧。标记可以选择文本通信中不使用的任何字符。然而现在, 我们传输其他如图片、音频、视频等的信息, 标记所使用的任何模式可能是这些信息的一部分, 如果接收方恰巧在数据的中段接收到这一模式, 它会认为已经到了帧的结束处了。为了解决这个问题, 在面向字符成帧中使用了字节填充 (byte-stuffing) 策略。字节填充 (或字符填充) 是指如果存在与标记相同模式的字符的话, 便在帧的数据部分填入一个特殊的字节。数据段被填入一个额外的字节, 这个字节通常称为换义字符 (escape character, ESC), 它有预定义的位模式。当接收方遇到换义字符时, 便会将它从数据段移走, 并仍将下一字符作为数据处理, 而不是当作分界的标记。

通过换义字符来进行字节填充允许标记出现在帧的数据段中, 但也产生了另一问题。如果紧随标记, 文本包含了一个或多个换义字符该怎么办? 接收方移走了换义字符, 但保留了标记, 会被错译为帧的末端。为了解决这一问题, 作为文本一部分的换义字符必须能被另一个换义字符标注。换言之, 如果换义字符是文本的一部分, 那么要增加额外的换义字符来表明第二个换义字符是文本的一部分。图11.2说明了这一情况。

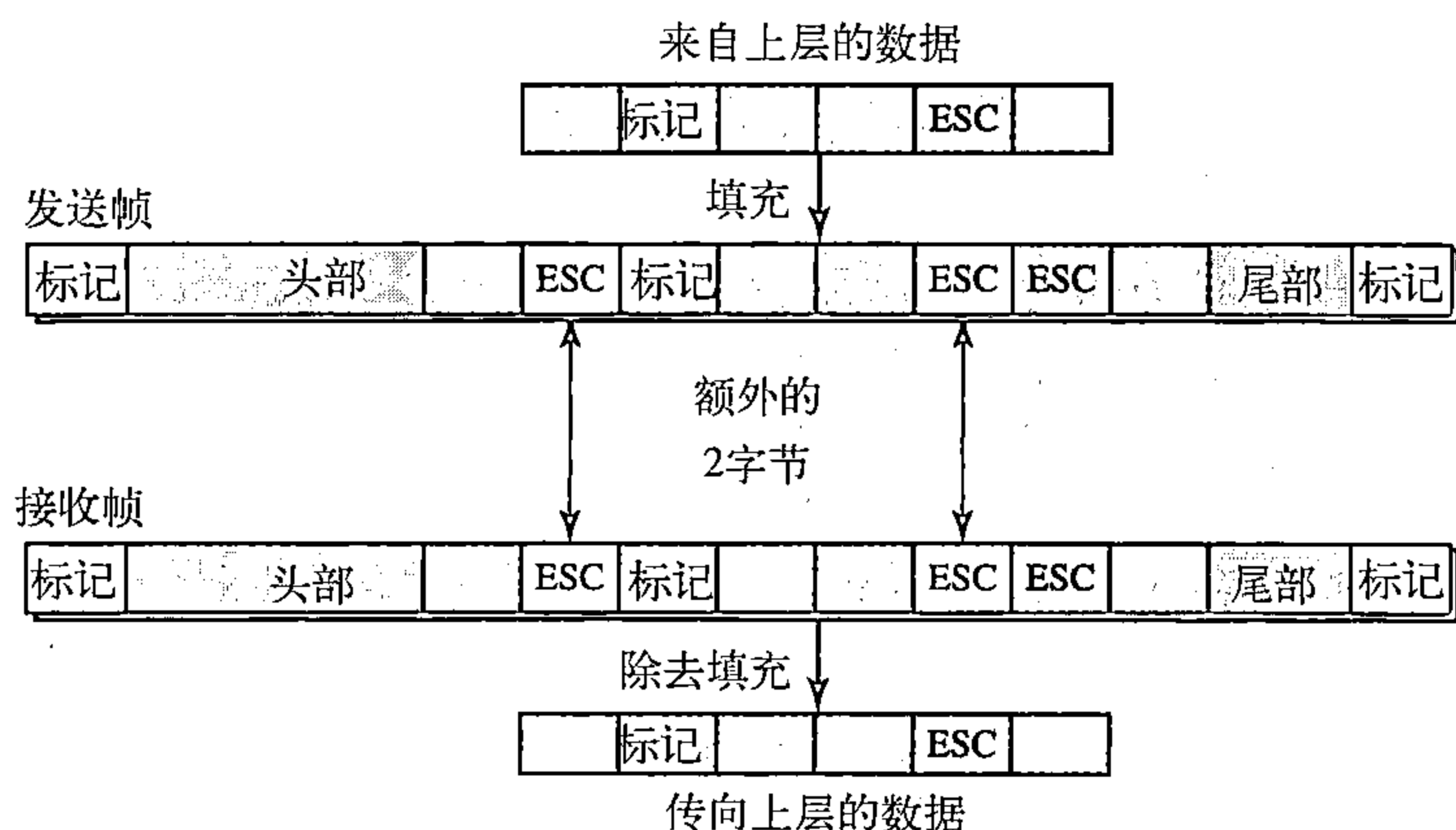


图11.2 字节填充和移除

字节填充是当文本中存在一个标记或换义字符时, 添加一个额外字节的过程。

面向字符协议在数据通信中产生了另一个问题。如今广泛应用的编码系统, 例如Unicode有16位和32位字符, 就会与8位字符产生冲突。所以我们可以说, 以下将讨论的面向位协议将是大势所趋。



### 面向位协议

在面向位协议 (bit-oriented protocol) 中, 帧的数据段是一个位的序列, 将由上层翻译成文本、图片、音频、视频等等。然而, 除了头部 (也可能是尾部) 之外, 我们仍然需要一个分隔符将帧与帧区分开来。多数协议使用一个特殊的8位模式的标记01111110作为分隔符来标明帧的开始和结束, 如图11.3所示。

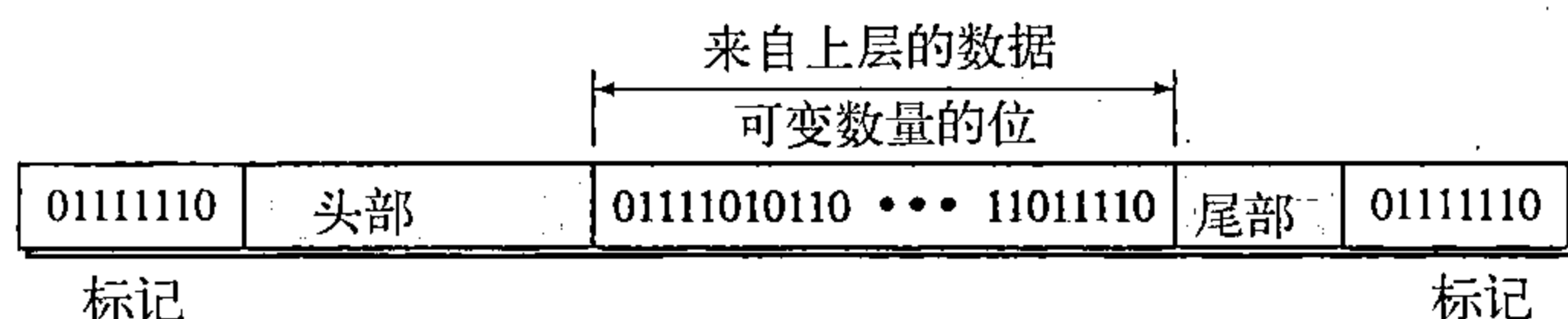


图11.3 面向位协议中的帧

此标记也会产生与面向字节协议相同的问题。也就是说, 当标记的模式出现在数据中时, 我们需要一定的信息告知接收方这并不是帧的结束。我们通过填充一个简单的位 (而非一个字节) 使得这个模式区别于标记, 这个策略就是位填充 (bit stuffing)。在位填充中, 当遇到1个0和5个连续的1, 便增加一个0。这个额外填充的位最终将被接收方移走。注意在1个0和5个连续的1后面填加1个额外的位而不管接下来那个位是什么, 这就保证了标记不会出现在帧中。

位填充是指在遇到1个0后面紧跟着5个1时便添加一个0的过程, 使得接收方不会误认01111110是一个标记。

图11.4说明了发送方填充位和接收方移走位的过程。注意, 当碰到在5个1以后如果是0, 我们仍然填充0。这个0将会被接收方移走。

这就意味着如果有类似标记的模式01111110出现在数据中时, 它会变成011111010 (填充后), 而不会被接收方误认为是一个标记。而真正的标记01111110是不会被发送方填充的, 故而能被接收方正确地确认。

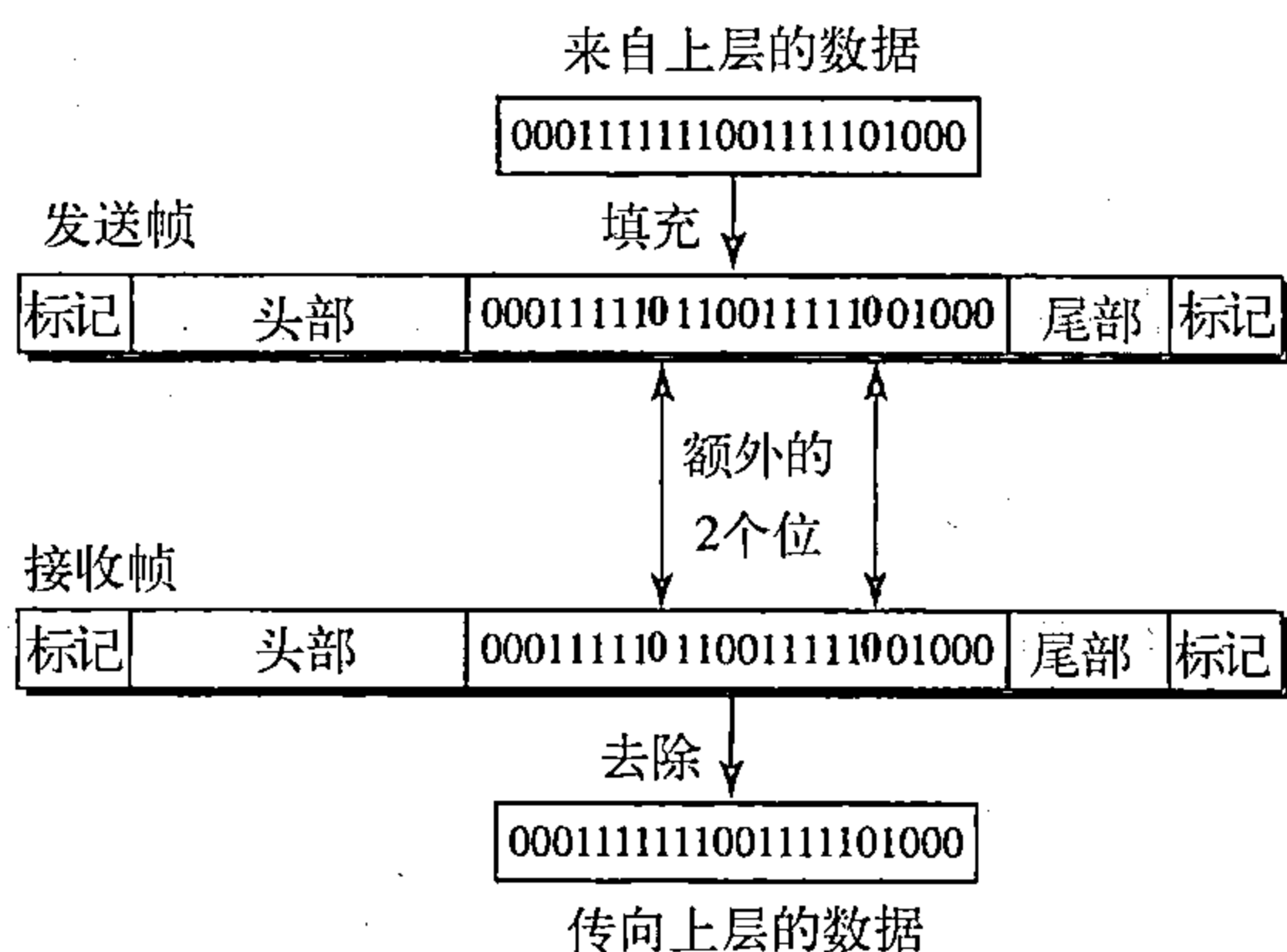


图11.4 位填充和移除

## 11.2 流量控制和差错控制

数据通信至少需要两个设备一起工作, 一个发送而另一个接收。就算是如此基本的过程也需要大量的协调来实现互相理解的交换。

数据链路层最重要的职能就是流量控制 (flow control) 和差错控制 (error control)。确切地说, 这些功能被视为数据链路控制 (data link control)。

### 11.2.1 流量控制

流量控制是数据链路层的一项重要职能, 它在接收确认前协调发送的数据数量。在大多数协议中, 流量控制是一系列程序, 告知发送方在等到来自接收方的确认之前它能传输多少数据。任何接收设备都有处理进入数据的速度限制和存储进入数据的容量限制。接收设备必须能够在达到这些限制前提示发送设备, 并要求传输设备发送的帧少一些或者暂时停止发送。进入的数据必须经过校验和处理才能使用。这些过程通常比传输速度慢。因此, 每个接收设备都有一块存储区, 称之为缓冲区, 存储进入的数据直到它们被处理。如果缓冲区也装满了, 接收方必须能告知发送方停止传输直到它能够再次接收为止。



流量控制就是一系列程序，用来限制发送方在等到确认之前发送的数据数量。

### 11.2.2 差错控制

差错控制包括差错检测和纠正。它使得接收方能够提示发送方在传输过程中有帧的丢失或破坏，并协调发送方重新传输这些帧。在数据链路层，术语差错控制通常指差错检测和重传的方法。在数据链路层中，差错控制一般容易实现：在交换的任何时刻检测到一个差错，就要重传这个出错的帧。这个过程被称之为自动重复请求（automatic repeat request, ARQ）。

数据链路层的差错控制基于自动重复请求，即重传数据。

## 11.3 协议

现在我们来看看数据链路层如何将成帧、流量控制和差错控制结合起来，从而实现节点对节点的数据传输。在软件中，通常使用一种通用程序语言实现协议。为了使我们的讨论不受程序语言的影响，我们对每一协议用伪码进行编写，从而主要关注程序本身而非研究程序语言的规则细节。

我们在此讨论的协议与在真实网络中使用的协议是有区别的。此处我们讨论的协议是单向的，数据帧从一个节点，称之为发送方，传输到另一个节点，称之为接收方。虽然有些特殊的帧，我们称之为确认（acknowledgment, ACK）

和否定确认（negative acknowledgment, NAK），但为了实现流量控制和差错控制，这些帧可以反方向流动，但数据只能单方向流动。

在现实生活的网络中，数据链路协议是双向实现的：数据可以双向流动。在这些协议中，流量控制和差错控制的信息，诸如ACK和NAK通过捎带（piggybacking）技术被包含在数据帧中。因为双向协议要比单向协议复杂的多，我们只讨论后者，只要理解了它们，就可以扩展到双向协议了。我们把扩展留做练习。

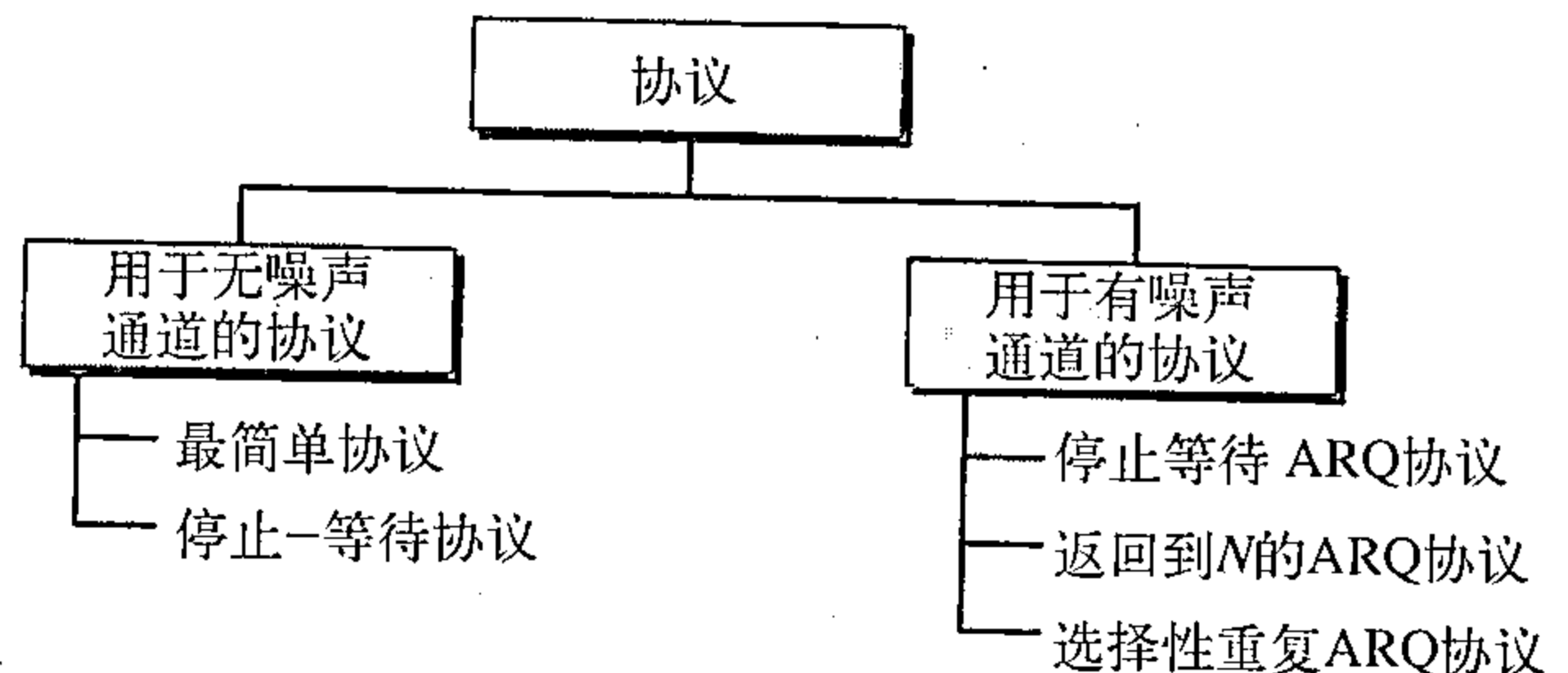


图11.5 本章所讨论的协议分类

## 11.4 无噪声通道

首先，我们假设有一种不会丢失帧、复制帧或损坏帧的理想通道，然后介绍此类通道的两种协议。第一种协议不使用流量控制，而第二种使用流量控制。当然，两者都不使用差错控制，因为，我们假设这种通道是完美的无噪声通道。

### 11.4.1 最简单的协议

我们讨论的第一个协议就称为最简单的协议（simplest protocol）而没有其他特定的命名，这个协议没有流量控制和差错控制。与其他我们将要在本章中讨论的协议一样，此协议是单向的，数据帧从发送方到接收方单方向传输。我们假定接收方可以即时处理它所接收的任何帧，而所花费的时间少到可以忽略不计。数据链路层中的接收方可以即时去除帧的头部并将数据分组传递到它的网络层，而网络层会即时收到分组。换言之，接收方永远不会因进入的帧而超载。



## 设计

在这种方案中无需流量控制。发送方站点的数据链路层从网络层获取数据，然后将数据成帧后发送。接收方站点的数据链路层从物理层接收到帧，从帧中取出数据，然后将数据传输到网络层。数据链路层的发送方和接收方为其网络层提供传输服务。数据链路层使用其物理层提供的服务（例如发信号、多路复用等）进行位的物理传输。图11.6给出了一个设计。

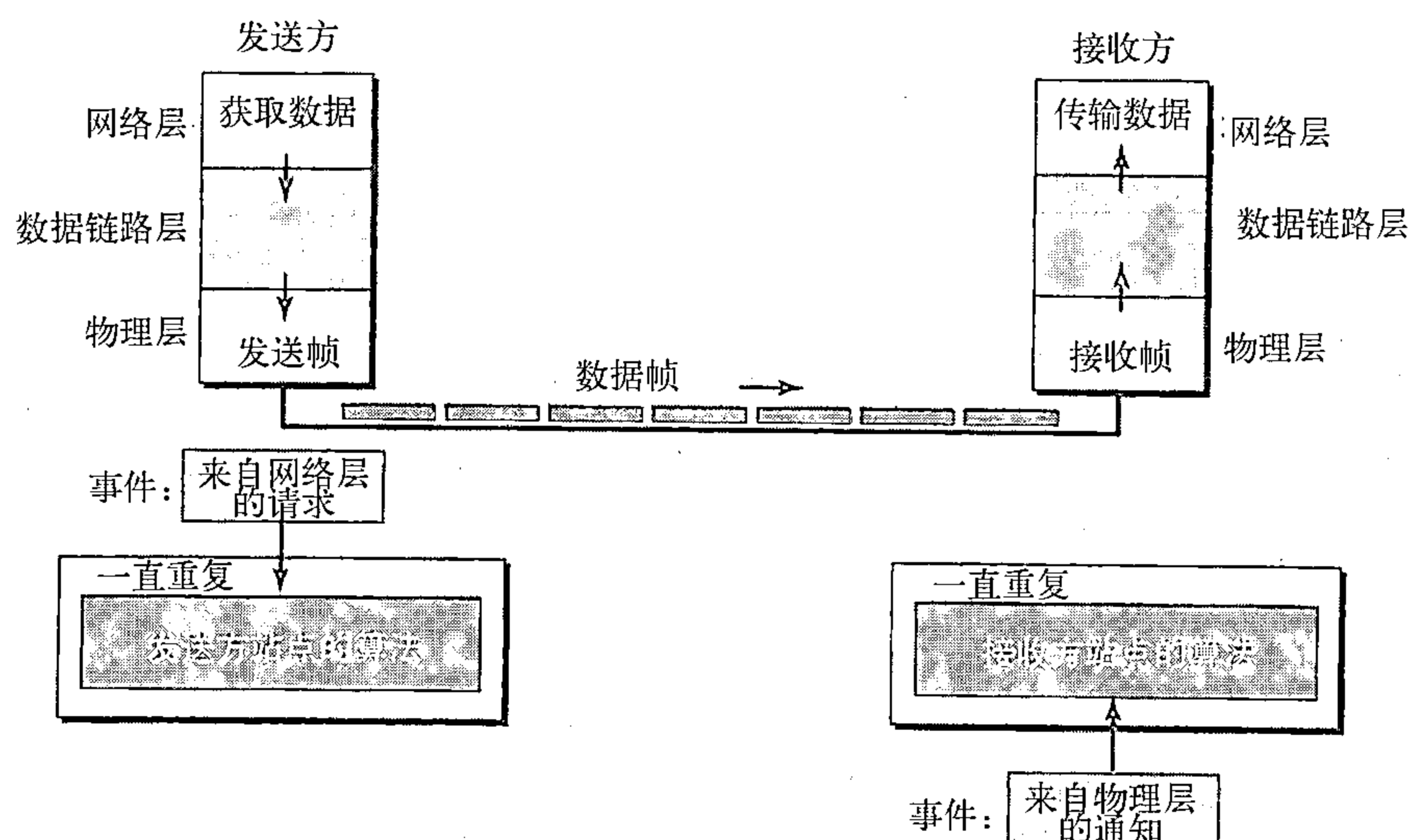


图11.6 没有流量控制和差错控制的最简单协议的设计

我们需要精心设计数据链路层所使用的程序。发送方站点直到其网络层有数据包要发送时才发送帧。接收方站点直到有帧到达才将数据分组传递给它的网络层。当协议用程序实现时，我们就要介绍协议中的事件（event）概念。发送方的程序不断地运行着，直到有来自网络层的请求才会有行为。接收方的程序也不断地运行着，直到有来自物理层的通知才会有行为。两端的程序都不断地运行，因为它们不知道相关事件何时发生。

## 算法

算法11.1给出了发送方的程序。

算法11.1 在最简协议中发送方站点的算法

```

1 while(true)                //不断重复
2 {
3   WaitForEvent();           //当一个事件出现时，唤醒它
4   if(Event(RequestToSend))  //有一个要发送的分组
5   {
6     GetData();
7     MakeFrame();
8     SendFrame();             //发送帧
9   }
10 }
```

分析 该算法有一个无限的循环，也就是说当程序一旦启动，第3行到第9行就会一直重复。该算法是事件驱动型的，也就是说它休眠（第3行）直到一个事件唤醒它（第4行）。这意味着第3行与第4行之间的执行存在不确定的时间段；它们之间存在缺口。当事件，即网络层发出的请求发生时，第6行到第8行被执行。然后，程序开始重复循环，它又在第3行休眠直到下一个事件发生时。我们为主要过程编写了伪代码，而并没有说明GetData、MakeFrame和



SendFrame这些模块的细节。GetData() 指从网络层获取数据包，MakeFrame() 在数据包中添加头部和分界的标记形成帧，SendFrame() 则将帧传输到物理层。

算法11.2给出了接收方站点的程序。

算法11.2 在最简协议中接收方站点的算法

```

1 while(true)                                //不断重复
2 {
3     WaitForEvent();                          //当一个事件出现时，唤醒它
4     if(Event(ArrivalNotification)) //数据帧到达
5     {
6         ReceiveFrame();
7         ExtractData();
8         DeliverData();                      //将数据发送给网络层
9     }
10 }

```

分析 除了帧和数据的传输方向是向上的以外，该算法和算法11.1的格式相同。此处的事件是数据帧的到达。该事件发生后，通过使用ReceiveFrame()语句，数据链路层从物理层获取帧，通过使用ExtractData()语句从帧中取出数据，并通过使用DeliverData()语句将数据传输到网络层。这里，我们也使用事件驱动型算法，因为该算法无法知晓帧何时会到达。

#### 例11.1

图11.7例举了一个使用最简协议通信的例子。这个例子非常简单。发送方发送一个帧序列而不用考虑接收方。为了发送三个帧，在发送方站点和接收方站点各发生了三个事件。注意图中倾斜的长方块表示数据帧，方块的高度定义为帧中第一位和最后一位传输的时间差。

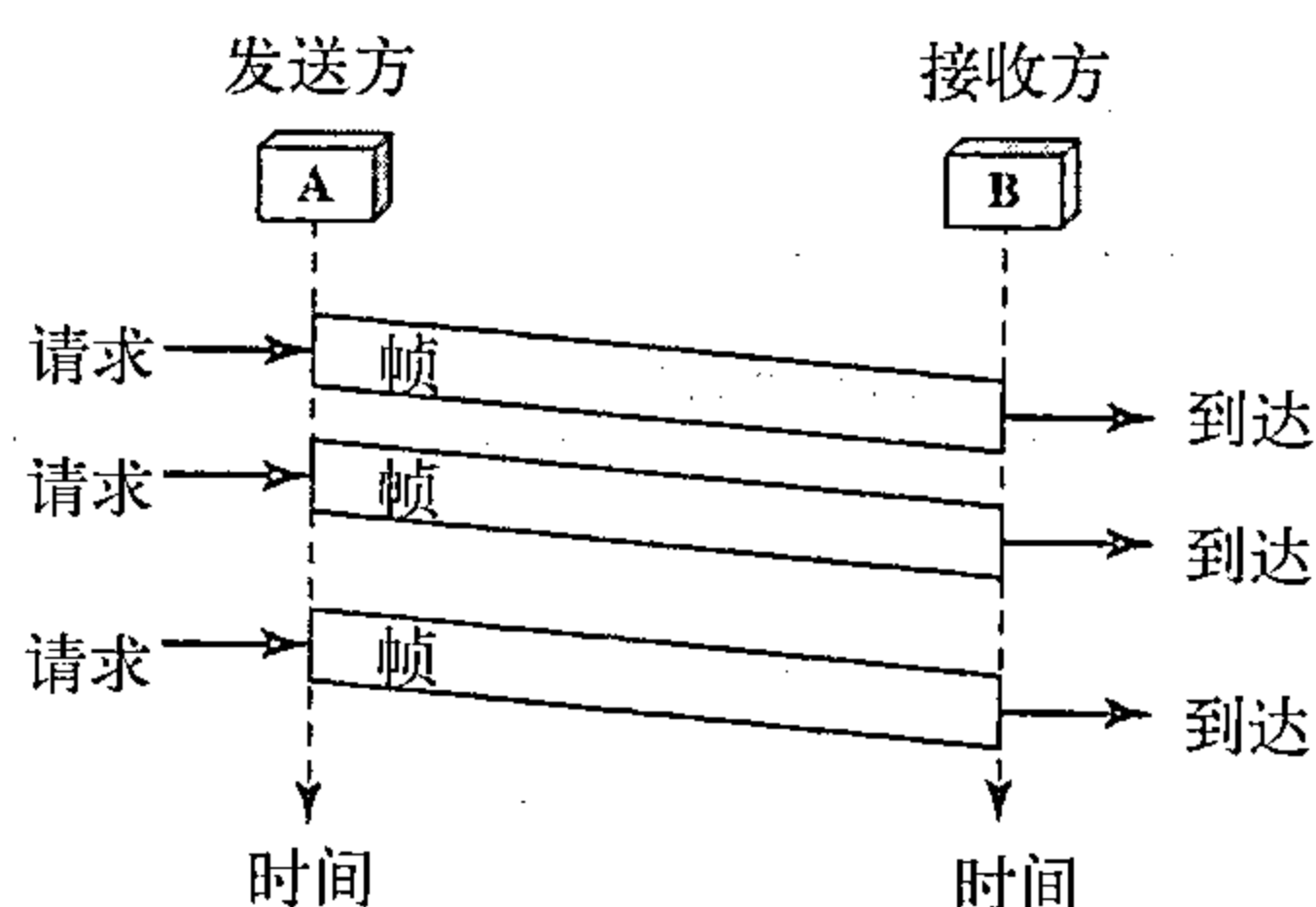


图11.7 例11.1的流量图

#### 11.4.2 停止等待协议

如果到达接收方的数据帧的速度大于它们能被处理的速度，帧在使用前必须被存储。通常，接收方没有足够的存储空间，特别是它将接收来自多个源的数据时。这样将导致帧被丢掉或者服务被拒绝。为了避免接收方出现帧超负荷，我们要想办法让发送方减缓速度。也就是说接收方必须给发送方反馈信息。

我们现在讨论的协议称为停止等待协议（stop-and-wait protocol），因为当发送方发出一个帧后必须停下来直到接收到来自接收方的确认（可以继续了），然后再发送一个帧。这里，数据帧仍是单向通信的，除了辅助的ACK帧（确认的简单标记）从另一端传输过来。我们在前面的协议中加入流量控制。

#### 设计

图11.8描述了该过程。将该图与图11.6对比，我们可以看到前向通道（从发送方到接收方）和反向通道的数据流。在任何时刻，不是在前向通道有一个数据帧，就是在反向通道中有一个ACK帧。因此，我们需要一个半双工的链路。

#### 算法

算法11.3是发送方站点的算法。



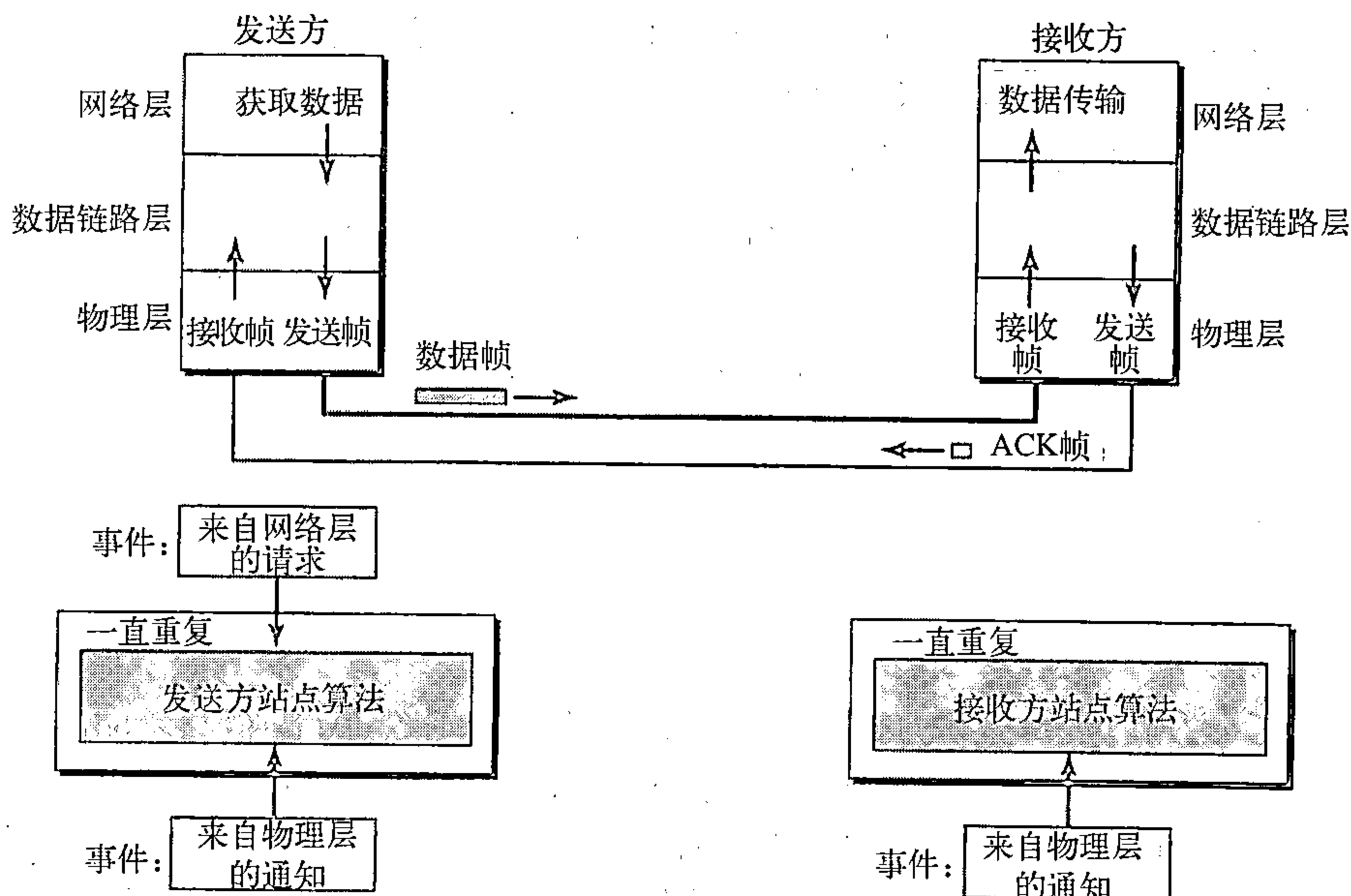


图11.8 停止等待协议的设计

算法11.3 停止-等待协议中发送方算法

```

1 while(true)                                //不断重复
2   canSend = true                             //允许发送第一个帧
3   {
4     WaitForEvent();                          //当一个事件出现时，唤醒它
5     if(Event(RequestToSend) AND canSend)
6     {
7       GetData();
8       MakeFrame();
9       SendFrame();                           //发送数据帧
10      canSend = false;                       //不能发送直到ACK到达
11    }
12    WaitForEvent();                          //当一个事件出现时，唤醒它
13    if(Event(ArrivalNotification))           //ACK到达
14    {
15      ReceiveFrame();                        //接收ACK帧
16      canSend = true;
17    }
18  }

```

分析 这里会发生两个事件：来自网络层的请求或来自物理层的通知。对这些事件的响应必须是交替的。换言之，当发送一个帧后，算法必须忽略另一个网络层的请求直到该帧被确认。我们知道两个到达事件不会一个紧接着一个发生，因为该通道是无差错的且不能复制帧。但是，网络层发送的请求可以一个紧接着一个发生而中间无需有到达事件的发生。所以，我们要在一定程度上阻止数据帧的即时发送。虽然有很多方法，但我们使用简单的变量 *canSend*，该变量既可以是正确 (true) 也可以是错误 (false)。当一个帧被发送，该变量被设置为错误，以表示不能发送新的网络请求直到 *canSend* 变为正确。当一个ACK帧被接收，*canSend* 便被设置为正确，以允许下一个帧的发送。

算法11.4 描述了接收方站点的程序。

分析 除了一处不同外，这个算法与算法11.2十分相似。当数据帧到达后，接收方发送一个ACK帧（第9行）以确认接收到并允许发送方发送下一个帧。



算法11.4 停止等待协议中接收方站点算法

```

1 while(true)                                //不断重复
2 {
3   WaitForEvent();                          //当一个事件发生时，唤醒它
4   if(Event(ArrivalNotification))           //数据帧到达
5   {
6     ReceiveFrame();
7     ExtractData();
8     Deliver(data);                          //将数据发送给网络层
9     SendFrame();                            //发送一个ACK帧
10  }
11 }

```

## 例11.2

图11.9 例举了一个使用停止-等待协议的例子。这个例子仍然很简单。发送方发送一个帧后要等待接收方的反馈。当ACK帧到达后，发送方发送下一个帧。注意，该协议中发送两个帧涉及了发送方的四个事件和接收方的两个事件。

## 11.5 有噪声通道

虽然停止-等待协议给出了我们如何在程序中增加流量控制的概念，但是无噪声通道是不存在的。我们能忽略差错（就想我们时常做的那样），或者我们需要为我们的协议增加差错控制。这部分我们讨论三个使用差错控制的协议。

## 11.5.1 停止等待自动重复请求

第一个协议称为停止等待自动重复请求（Stop-and-Wait ARQ），是在停止等待协议中增加了一个简单的差错控制机理。让我们来看看这个协议如何检测并纠正差错。

为了检测和纠正被损坏的帧，我们需要给数据帧加上冗余的位（见第10章）。当帧到达接收方站点时，它被检测，如果它被损坏了，它会被无声地丢弃。此协议中差错检测由接收方的静音来证明。

对丢失帧的处理远比对损坏帧的处理要难的多。在我们先前的协议中，我们没有办法确认一个帧。接收到的帧可以是正确的，或者复制的，也可以是无序的。解决的途径是为帧编号。当接收方接收到一个无序的数据帧时，就意味着这些帧或者是丢失的或者是被复制的。

在此协议中，要重发被损坏的帧和丢失的帧。当有差错时，如果接收方没有响应，那么发送方如何知道要重发哪个帧呢？为了解决这个问题，发送方保留已发送帧的副本。与此同时，它启动一个定时器。如果定时器到时且没有收到已发送帧的ACK，就要重发该帧，保留副本并重启定时器。既然该协议使用停止等待机理，那么只有一个特定帧需要ACK，尽管网络中会存在同一个帧的多个副本。

在停止等待自动重复请求协议中，差错的检测由保留已发送帧的副本并当定时器到时重传这个帧来实现。

既然一个ACK帧同样会被损坏或丢失，它也需要冗余位和一个序列号。此协议中的ACK帧有一个序列号字段。此协议中，发送方只要简单的丢弃一个损坏的ACK帧或者忽略一个无序的帧。

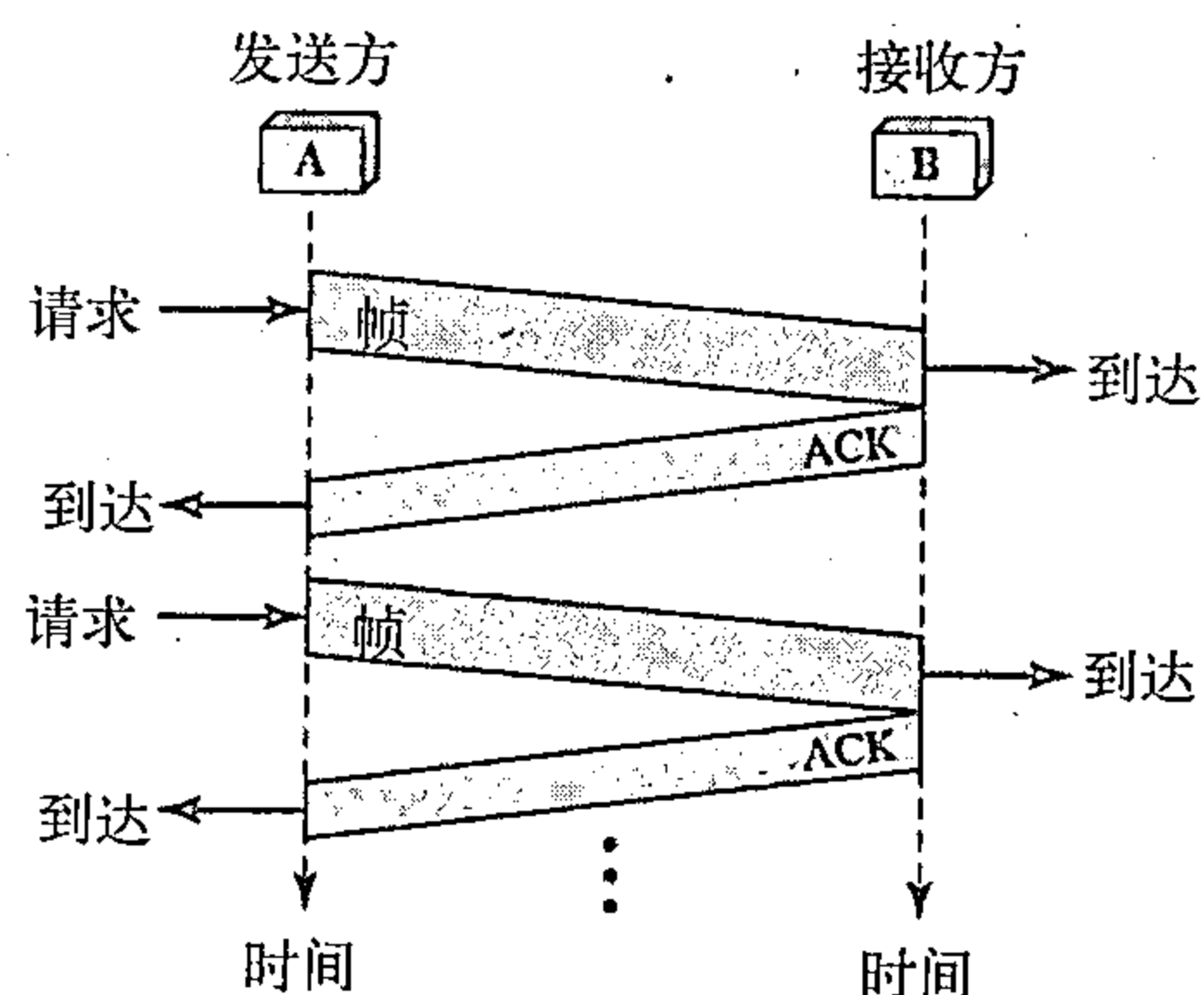


图11.9 例11.2的流量图



### 序列号

正如我们讨论的那样，此协议要求必须对帧进行编号。这通过使用序列号 (sequence number) 来完成。数据帧要增加一个字段来容纳帧的序列号。

序列号的排列是一个值得考虑的重要问题。既然我们希望帧的长度越小越好，我们寻找能提供无歧义通信的最小范围。序列号当然是可以循环的，例如，我们决定字段 $m$ 位长，序列号从0开始，到 $2^m-1$ ，再开始重复。

让我们来推出所需序列号的范围。假定我们使用 $x$ 作为一个序列号，接下来我们只需用 $x+1$ ，而没有必要用 $x+2$ 。为了加以演示，假定发送方已经发送了序列号为 $x$ 的帧，那么将会发生三种情况。

1. 帧安全到达接收方，接收方发送确认。确认到达发送方，促使发送方发送下一个序列号是 $x+1$ 的帧。

2. 帧安全到达接收方，接收方发送确认，但是确认被损坏或丢失了。在超时后发送方重发帧 (编号 $x$ )。注意此处的帧是重复帧。接收方可以确认这一事实，因为它预期帧 $x+1$ 到达但接收到的却是帧 $x$ 。

3. 帧被损坏或者没有到达接收方时，在超时后发送方重发帧 (编号 $x$ )。

我们可以看到有必要使用序列号 $x$ 和 $x+1$ ，因为接收方需要区分情况1和情况2。但是没有必要将帧编号为 $x+2$ 。在情况1中，帧可以再编号为 $x$ ，因为帧 $x$ 和 $x+1$ 被确认而且在任何一端都没有歧义。在情况2和3中，新的帧是 $x+1$ ，不是 $x+2$ 。如果只需要 $x$ 和 $x+1$ ，我们可以使 $x=0$ 及 $x+1=1$ 。这就意味着序号是01010等。这个模式熟悉吗？这就是我们在第10章看到的模2运算。

在停止等待ARQ中，我们使用序列号给帧编号。该序列号基于模2运算。

### 确认号

既然序列号必须既适用于数据帧，也要适用于ACK帧，那么我们使用这个约定：确认号总是告知接收方期待的下一个帧的序列号。例如，如果帧0安全到达，接收方发送一个带有确认1的ACK帧 (表示下一个期待收到的是帧1)。如果帧1安全到达，接收方则发送一个带有确认0的ACK帧 (表示下一个期待收到的是帧0)。

在停止等待ARQ中，确认号总是以模2运算宣布期待收到的下一个帧的编号。

### 设计

图11.10描述了停止等待ARQ协议的设计。发送设备保留最后传输的帧的副本直到接收到该帧的确认。一个数据帧使用一个序列号，一个ACK帧使用一个确认号。发送方有一个控制变量，我们称为 $S_n$  (发送方，下一个要发送的帧)，它保存下一个将被发送的帧的序号 (0或1)。

接收方有一个控制变量，我们称之为 $R_n$  (接收方，期待收到的下一个帧)，它保存下一个期待收到帧的编号。当一个帧被发送， $S_n$ 值递增 (模2)，也就是说如果它是0，它会变成1；如果是1则会变成0。当一个帧被接收时， $R_n$ 值递增 (模2)，也就是说如果它是0，它会变成1；如果是1则会变成0。发送方站点将有三个事件发生，接收方站点发生一个事件。变量 $S_n$ 指向的时隙与已经发送的帧的序列号相匹配，但没有被确认； $R_n$ 指向的时隙与期待收到的帧的序列号相匹配。

### 算法

算法11.5是发送方站点的算法。



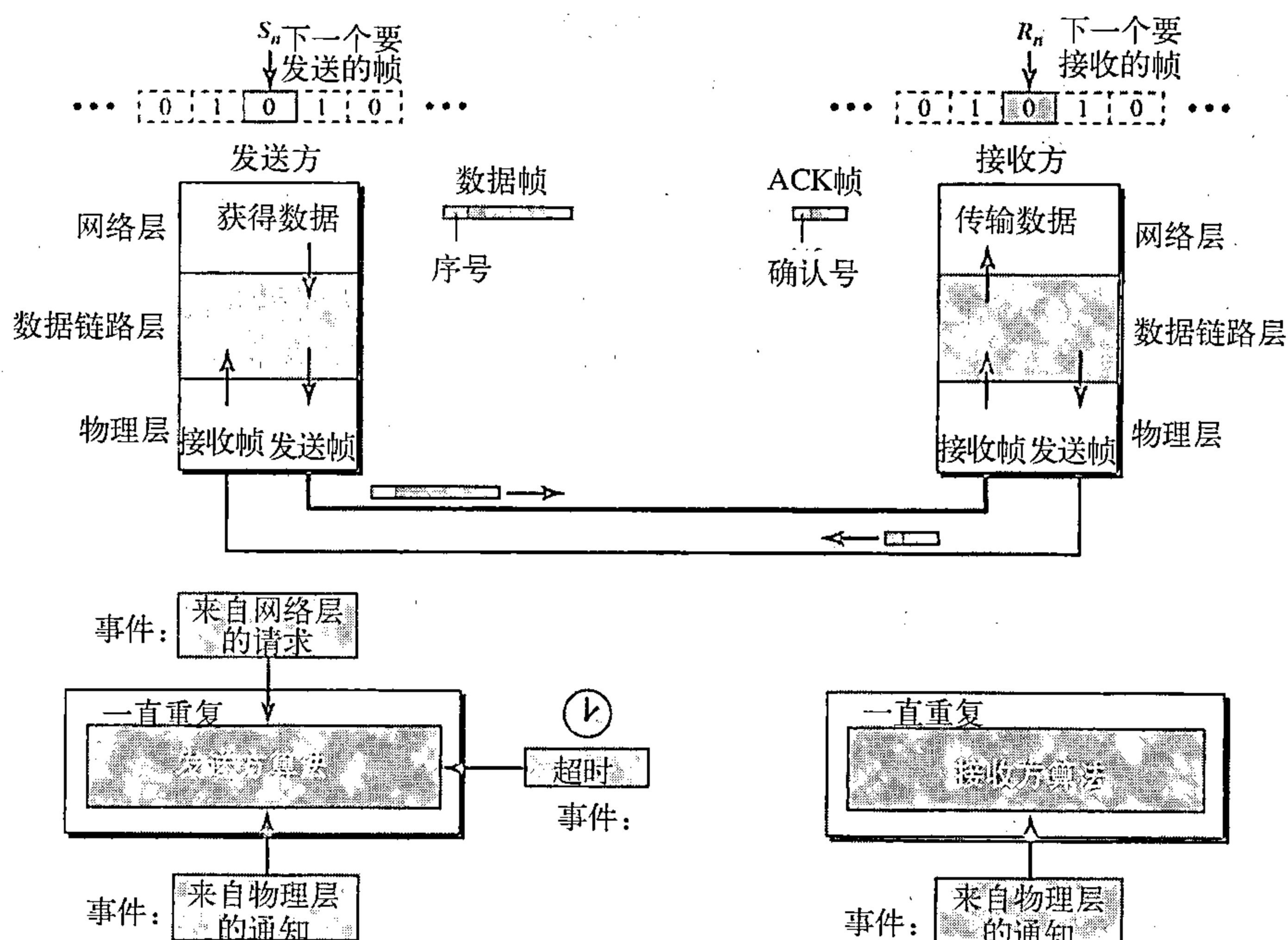


图11.10 停止等待ARQ协议设计

算法11.5 停止等待ARQ协议中发送方站点的算法

```

1  Sn = 0; //帧0被最先发出
2  canSend = true; //允许最初请求
3  while(true) //无休止重复
4  {
5      WaitForEvent(); //当一个事件发生时, 唤醒它
6      if(Event(RequestToSend) AND canSend)
7      {
8          GetData();
9          MakeFrame(Sn); //顺序号为Sn
10         StoreFrame(Sn); //保留副本
11         SendFrame(Sn);
12         StartTimer();
13         Sn = Sn + 1;
14         canSend = false;
15     }
16     WaitForEvent(); //休眠
17     if(Event(ArrivalNotification) //ACK帧到达
18     {
19         ReceiveFrame(ackNo); //接收ACK帧
20         if(not corrupted AND ackNo == Sn) //有效ACK帧
21         {
22             Stoptimer();
23             PurgeFrame(Sn-1); //不需要的副本
24             canSend = true;
25         }
26     }
27
28     if(Event(TimeOut) //超时
29     {
30         StartTimer();
31         ResendFrame(Sn-1); //发送副本检查
32     }
33 }

```

分析 我们首先注意到 $S_n$ , 它是下一个将要发送的帧的序列号。该变量只初始化一次



(第1行), 但每次发送一个帧后(第13行), 它便递增为下一个帧做准备。然而, 既然这是模2运算, 那么序列号就是0、1、0、1等。注意第一个事件的进程(SendFrame, SotreFrame, PurgeFrame), 使用 $S_n$ 来定义被发送的帧。我们至少需要一个缓冲区来保存这个帧直到我们能确认它已被安全接收。第10行指出在帧被发送前, 它被保存了。副本用来重发一个损坏或者丢失的帧。我们仍然使用变量canSend在前一个帧安全接收之前防止网络层提出请求。如果这个帧没有损坏, 且ACK帧的确认号与下一个将要发送的帧的序列号匹配, 我们便停掉定时器并清除保存的数据帧副本。否则, 我们只要忽略这个事件并等待下一个事件发生就行了。每一个帧被发送后, 一个定时器开始计时。当定时器超时(第28行), 帧被重发, 且定时器被重置。

算法11.6 显示了接收方站点的程序。

算法11.6 停止等待ARQ协议中接收方站点的算法

---

```

1   $R_n = 0;$  //帧0有望首先到达
2  while(true)
3  {
4      WaitForEvent(); //当一个事件发生时, 唤醒它
5      if(Event(ArrivalNotification)) //数据帧到达
6      {
7          ReceiveFrame();
8          if(corrupted(frame));
9          sleep();
10         if(seqNo ==  $R_n$ ) //有效数据帧
11         {
12             ExtractData();
13             DeliverData(); //传送数据
14              $R_n = R_n + 1;$ 
15         }
16         SendFrame( $R_n$ ); //发送ACK帧
17     }
18 }

```

---

分析 这显然与算法11.4不同。首先, 所有已到达数据帧的损坏被忽略了。当帧的序列号是期待收到的那个( $R_n$ )时, 帧才被接收, 数据才被传输到网络层,  $R_n$ 值增加。然而, 这里有一个微小的差别。即使数据帧的序列号与下一个期待的帧的序列号不匹配, 也会发送一个ACK帧给发送方。然而这个ACK只是再确认前一个ACK, 而不是确认接收到的帧。这样做是因为接收方认为先前的ACK有可能已经丢失了, 接收方便发送一个副本。重发的ACK可能在时钟超时之前解决问题。

### 例11.3

图11.11给出了一个停止等待ARQ协议的例子。帧0被发送并被确认。帧1丢失了并在超时之后被重发。重发的帧1被确认, 定时器便停止计时。帧0被发送并确认, 但是确认丢失了。发送方无法判断是帧被丢失了还是确认被丢失了, 因此超时后, 它重发帧0, 虽然帧0已被确认了。

### 效率

如果我们的通道粗且长的话, 那么前部分讨论的停止等待ARQ协议是十分低效的。粗指的是通道有很大的带宽, 长指的是往返延迟较长。这两者的乘积称为带宽延迟乘积(bandwidth-delay product), 正如我们在第3章中讨论的。我们可以把这种通道看作一根管道, 那么带宽延迟乘积就是以位为单位的管道容量。管道总是在那里, 如果我们不使用它, 那就是无效的。带宽延迟乘积是一种尺度, 用来度量我们在等待来自接收方的消息的同时所能发送到系统外的位数。



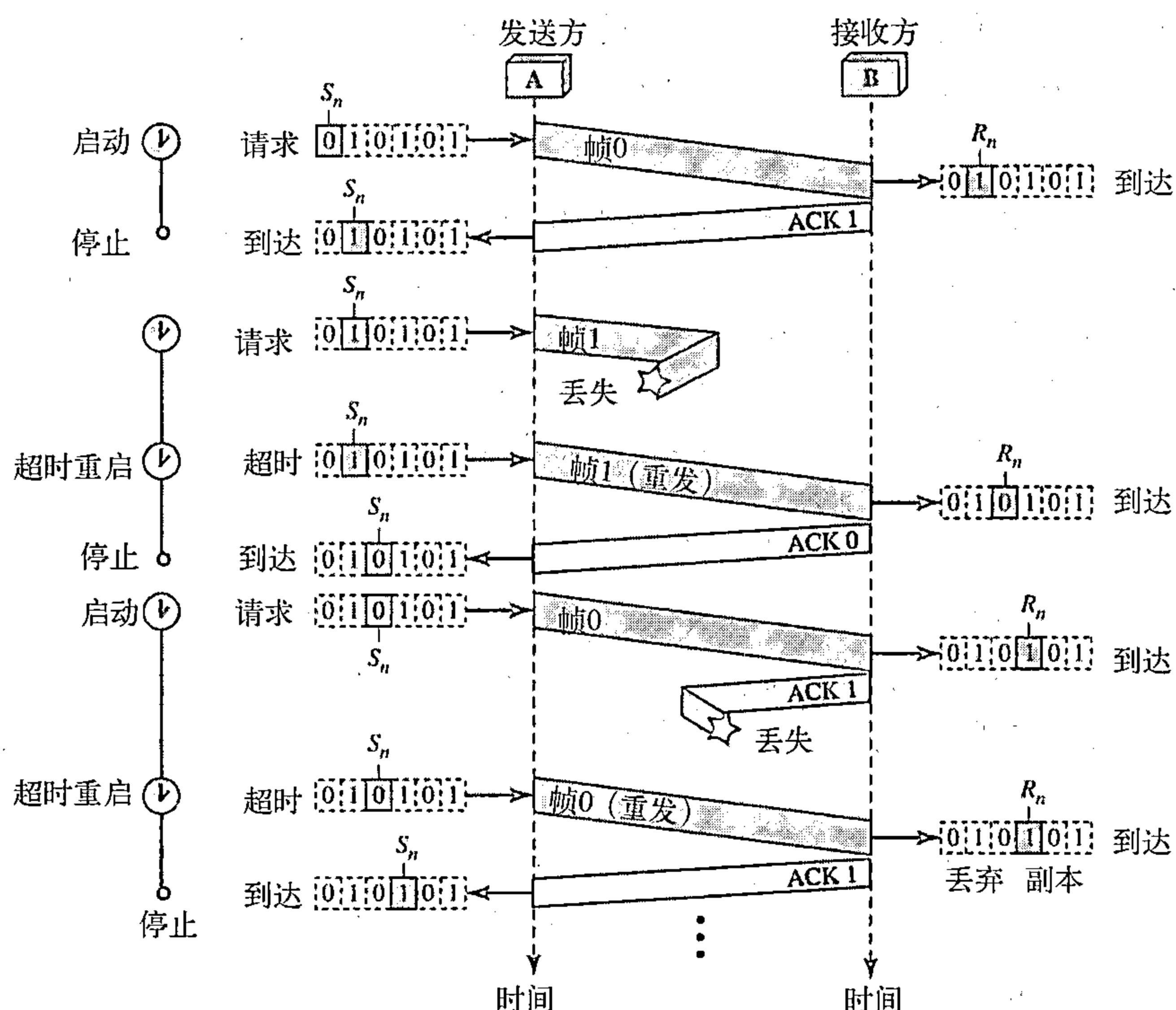


图11.11 例11.3的流量图

**例11.4**

假定，在一个停止等待ARQ系统中，带宽是1Mbps，且一个位往返一次是20微秒。那么带宽延迟乘积是多少？如果系统的数据帧的长度是1000位，那么链路的利用率是多少？

解：

带宽延迟乘积是

$$(1 \times 10^6) \times (20 \times 10^{-6}) = 20\,000 \text{ 位}$$

在数据从发送方到接收方再返回的期间，系统能够发送20 000位。然而，系统只能发送1 000位。我们因此说链路的利用率仅有1 000/20 000，即5%。正因为此，如果链路有高带宽或长延迟的话，使用停止等待ARQ协议是对链路能效的一个浪费。

**例11.5**

在例11.4中，如果我们拥有一种协议能在停止和担心确认之前发送15个帧，那么链路层的利用率是多少？

解：

带宽延迟乘积仍然是20 000位。在一个往返过程中，系统能发送15个帧或者15 000位，这意味着利用率是15 000/20 000，即75%。当然，如果存在损坏的帧，那么利用率就要低很多，因为这些帧要被重发。

**流水线操作**

在网络或者其他领域中，一项任务通常在前一项完成后开始，这就是流水线操作(pipelining)。在停止等待ARQ协议中不存在流水线操作，因为我们必须等待一个帧到达目的地并被确认后才能发送下一个帧。然而，流水线操作应用到了我们下面的两个协议中，因为



在我们得到关于前几个帧的消息前，就能发送后几个帧了。如果传输的位是巨大的，考虑到带宽延迟乘积，流水线操作改进了传输的效率。

### 11.5.2 回退N帧自动重发请求

为了提高传输的效率（填满管道），在等待确认时必须传输多个帧。换言之，在发送方等待确认时，我们需要让多个帧待处理来使通道忙碌。在这一节，我们讨论一个能够达到这个目的的协议，在下一节，我们将讨论另一个。

第一个协议称之为回退N帧自动重发请求（Go-Back-N Automatic Repeat Request）（该名称的基本原理会在后面说明）。在这个协议中，我们在接收到确认之前能发送几个帧，在确认到达之前我们会保存这些帧的副本。

#### 序列号

帧在发送站点按序编号。然而，因为我们要让每一个帧的头部中包含序列号，我们需要设置一个限制值。如果一个帧的头部允许序列号有 $m$ 位，序列号的范围就是0到 $2^m - 1$ 。例如，如果 $m$ 等于4，那么序列号只能是0到15中的数字。当然，我们可以重复序列号。因此，序列号是

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0,  
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ……………

换言之，序号是模 $2^m$

在回退N帧协议中，序列号是模 $2^m$ ，这里 $m$ 是以位为单位的序列号字段长度。

#### 滑动窗口

在此协议（和下一个协议）中，滑动窗口（sliding window）是一个抽象的概念，用来定义发送方和接收方关心的序列号范围。换言之，发送方和接收方只需处理可能的序列号部分。发送方关心的序列号范围称为发送滑动窗口（send sliding window），接收方关心的则称为接收滑动窗口（receive sliding window）。我们将对两者进行讨论。

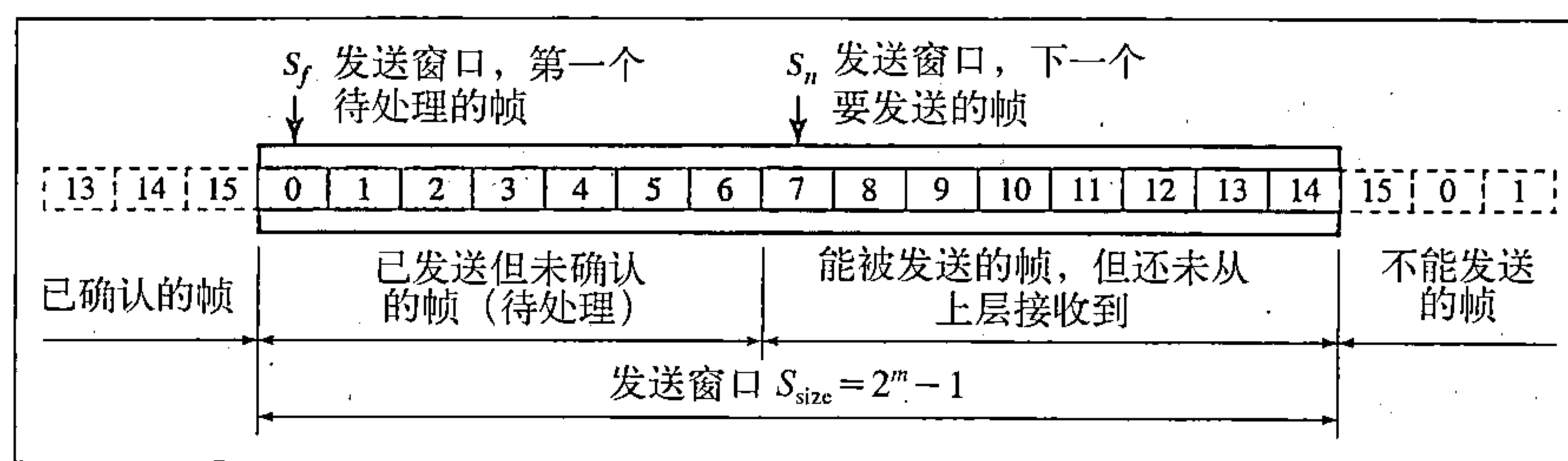
发送窗口是一个假想的盒子，包含了能传输的数据帧的序列号。在每一个窗口中，一部分序列号定义了已经发送的帧，另一部分定义了能够被发送的帧。窗口的最大尺度是 $2^m - 1$ ，原因我们以后阐述。在这章中，我们固定窗口的大小，使之为最大值，但是在以后的章节中我们会看到一些协议有可变的窗口大小。图11.12给出了一个大小为15（ $m=4$ ）的滑动窗口。

在任何时刻窗口将可能适用的序号分成四部分。第一部分，从最左端到窗口的左墙，定义属于已经被确认的帧的序号。发送方不用担心这些帧，也不用保留它们的副本。第二部分，图11.12a的着色部分，定义了属于已发送但处于未知状态的帧的序号。发送方必须等待直到知道这些帧被接收或者被丢失。我们称这些帧为待处理的帧。第三部分，图中的空白部分，显示了那些能够被发送的帧的序号范围，但还没有从网络层接收到相对应的数据包。最后，第四部分显示了那些直到窗口滑动才能使用的序号，正如我们接下来看到的那样。

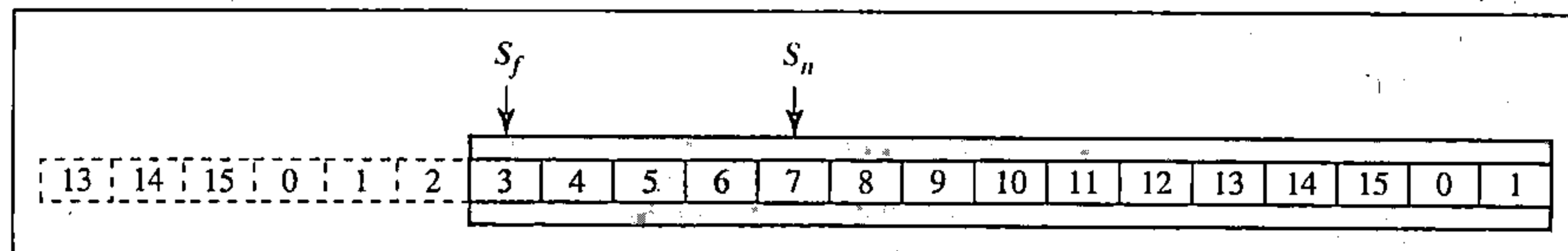
窗口本身是一个抽象体，在任何时刻用三个变量界定了它的大小和位置。我们称这些变量为 $S_f$ （发送窗口，第一个待处理的帧）， $S_n$ （发送窗口，下一个要发送的帧），和 $S_{size}$ （发送窗口，大小）。变量 $S_f$ 定义了第一个（最先）待处理帧的序号。变量 $S_n$ 定义了分配给下一个将发送帧的序号。最后，变量 $S_{size}$ 定义了窗口的大小，该变量在此协议中是固定的。

发送窗口是一个抽象概念，通过三个变量 $S_f$ 、 $S_n$ 和 $S_{size}$ 定义的一个大小为 $2^m - 1$ 的假想盒子。





a) 滑动前的发送窗口



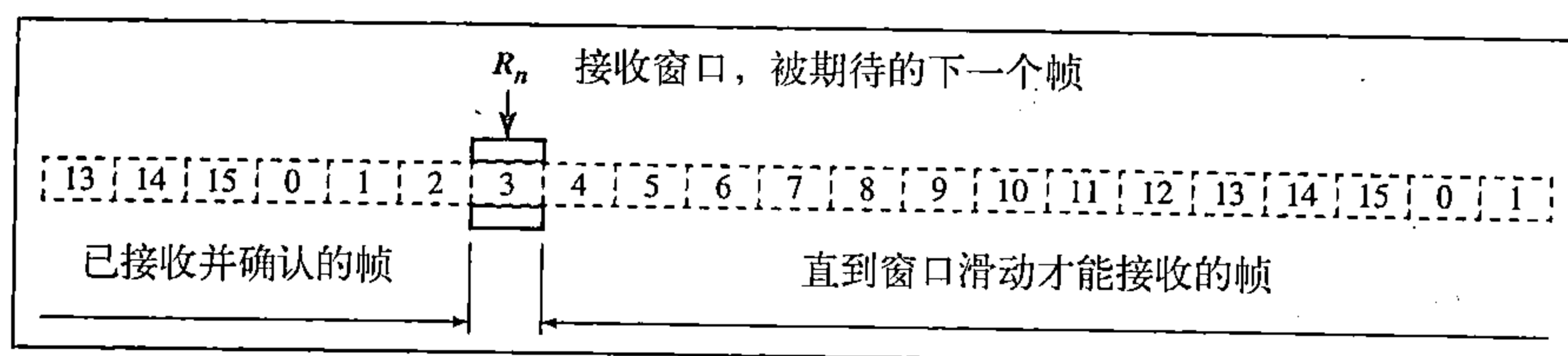
b) 滑动后的发送窗口

图11.12 回退N帧自动重发请求协议中的发送窗口

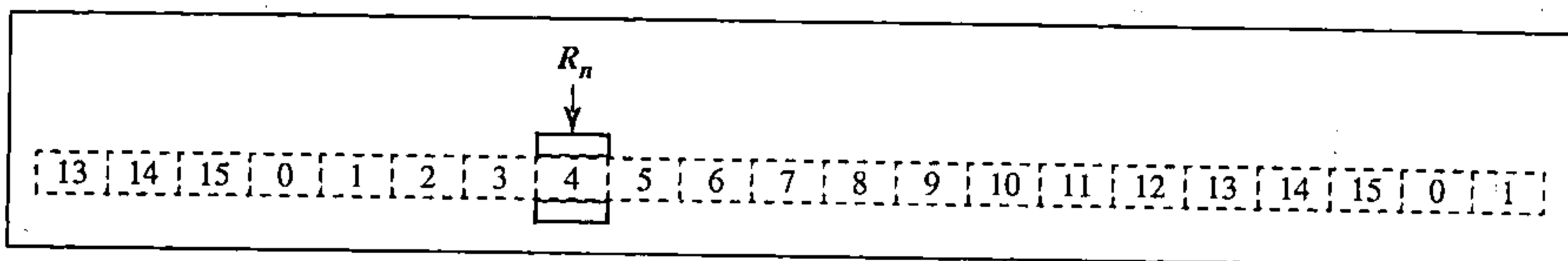
图11.12b描述了当另一端发来的确认到达时，一个发送窗口可以向右滑动一个或者多个时隙。正如我们不久可以看到的那样，在此协议中，确认是累加的，这意味着一个ACK帧可以确认一个以上的帧。在图11.12b中，帧0, 1和2已被确认，因此窗口向右滑动了三个时隙。注意 $S_f$ 的值是3，因为帧3现在是待处理的帧。

有效确认到达时，发送窗口能滑动一个或者多个时隙。

接收窗口确定所接收的数据帧时，发送正确的确认。接收窗口的大小始终是1。接收方总是在等待一个指定帧的到达。任何到达的失序帧都会被丢掉并要求重发。图11.3显示了接收窗口。



a) 接收窗口



b) 滑动后的窗口

图11.13 回退N帧自动重发请求协议中的接收窗口

接收窗口是一个抽象概念，通过唯一变量 $R_n$ 定义了一个大小为1的假想盒子。当一个正确的帧到达后，窗口滑动，但每次只滑动一个时隙。

注意：我们只需要一个变量 $R_n$ （接收窗口，期待的下一个帧）来定义这个抽象概念。窗口左边的序号属于那些已接收并确认的帧，右边的序号显示了那些不能被接收的帧。任何已收到帧的序号属于这两部分都会被丢弃。只有当一个帧的序号与 $R_n$ 匹配时，才被接收和确认。

接收窗口同样会滑动，但是每次只滑动一个时隙。当收到一个正确的帧（且每次只能收到一个帧）时，窗口才滑动。



### 定时器

虽然每一个已发送的帧都能有一个定时器，但在我们的协议中我们只用一个。原因是第一个待处理帧的定时器总是先到时，当这个定时器到时，我们发送所有的待处理帧。

### 确认

当一个帧安全有序到达时，接收方发送一个肯定的确认。如果一个帧被损坏了或者收到时次序颠倒了，接收方便不响应并将丢弃所有后来的帧直到它收到一个它所期待的帧。接收方的不响应使得不被确认的帧的定时器在发送方站点到时。这样，接下来会使发送方返回，并从定时器到时的那个帧开始重发所有的帧。接收方不需要确认每一个收到的帧。它能为多个帧发送一个累积的确认。

### 重发帧

当定时器到时的时候，发送方会重发所有待处理的帧。例如，假定发送方已经发送了帧6，但是帧3的定时器到时了。这就意味着帧3没有被确认，发送方就要返回并重发帧3，4，5和6。这就是这个协议称之为回退N帧自动重发请求协议的原因。

### 设计

图11.14描述了该协议的设计。正如我们可以看到的，多个帧能在前向方向传输，在反向传输多个确认。这个理念与停止等待ARQ协议类似，区别在于发送窗口允许我们传输与其时隙一样的帧。

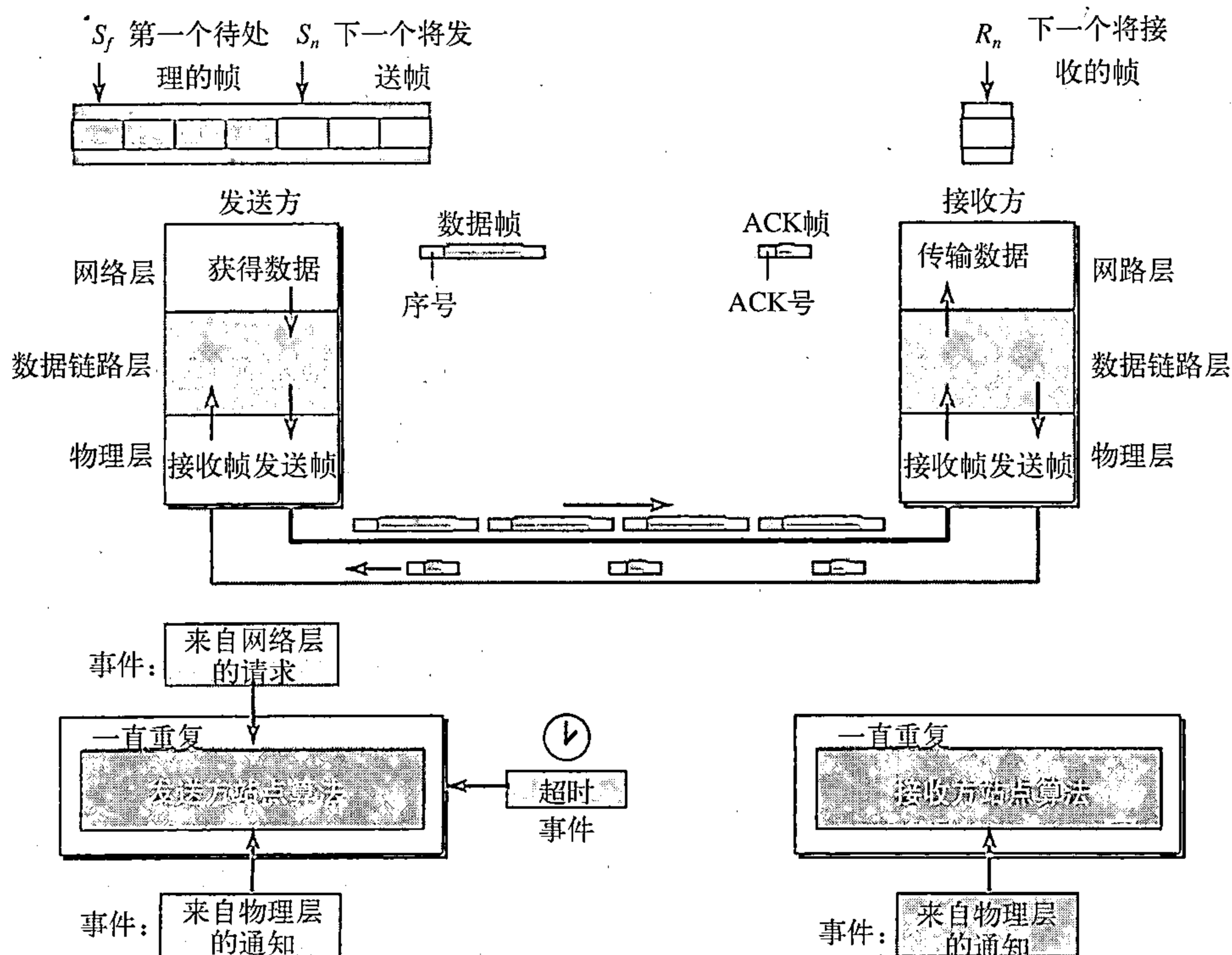


图11.14 到N步返回的自动重复请求协议的设计

### 发送窗口大小

我们现在可以阐述为什么发送窗口的大小必须小于 $2^m$ 了。作为一个示例，我们使 $m=2$ ，这就意味着窗口的大小是 $2^m-1$ ，即3。图11.15将一个大小为3的窗口和一个大小为4的窗口进



行了比较。如果窗口的大小为3（小于 $2^2$ ）且三个确认被丢失了，帧0的定时器到时，那么所有三个帧都要被重发。接收方现在期待的是帧3，而不是帧0，于是副本帧被正确地舍弃了。另一方面，如果窗口的大小是4（等于 $2^2$ ）且所有确认被丢失了，发送方就要重发帧0的副本。然而，这次接收方的窗口期待收到帧0，于是它接收了帧0，是下一周期中的第一个帧，而不是一个丢失帧的副本。这是一个差错。

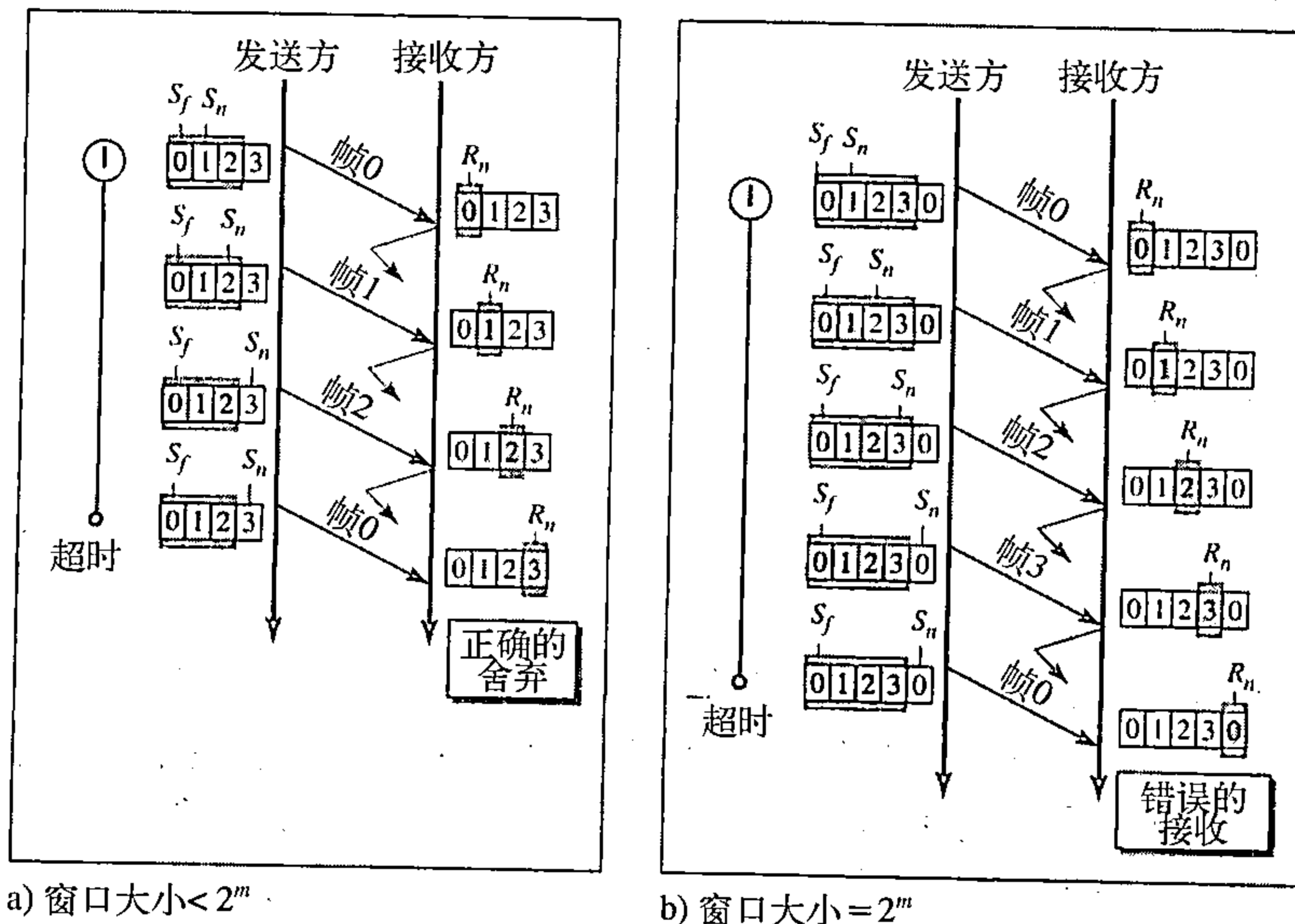


图11.15 在回退N帧请求重发协议中窗口的大小

在回退N帧自动重发请求协议中，发送窗口的大小必须小于 $2^m$ ，而接收窗口的大小始终是1。

### 算法

算法11.7是在回退N帧自动重发请求协议中发送方算法。

算法11.7 回退N帧自动重发请求的发送方算法

```

1   $S_w = 2^m - 1;$ 
2   $S_f = 0;$ 
3   $S_n = 0;$ 
4
5  while (true)                                //不断重复
6  {
7      WaitForEvent();
8      if(Event(RequestToSend))                //发送包
9      {
10         if( $S_n - S_f \geq S_w$ )                  //如果窗口满
11             Sleep();
12         GetData();
13         MakeFrame( $S_n$ );
14         StoreFrame( $S_n$ );
15         SendFrame( $S_n$ );
16          $S_n = S_n + 1;$ 
17         if(timer not running)
18             StartTimer();
19     }
20
21     if(Event(ArrivalNotification)) //ACK帧到达
22     {
23         Receive(ACK);

```



```

24     if(corrupted(ACK))
25         Sleep();
26     if((ackNo>Sf)&&(ackNo<=Sn)) //如果为有效
27     While(Sf <= ackNo)
28     {
29         PurgeFrame(Sf);
30         Sf = Sf + 1;
31     }
32     StopTimer();
33 }
34
35 if(Event(TimeOut)) //计时器超时
36 {
37     StartTimer();
38     Temp = Sf;
39     while(Temp < Sn);
40     {
41         SendFrame(Sf);
42         Sf = Sf + 1;
43     }
44 }
45 }

```

分析 该算法先初始化了三个变量。与停止等待ARQ协议不同，这个协议允许从网络层得到多个请求而不需要其他事件的发生。我们只需要确定窗口并没有满（第12行）。在我们的方式中，如果窗口满了，请求就会被忽视而网络层就要再尝试。有一些实现使用了其他的方法比如激活或者抑制网络层。对到达事件的处理远比之前的协议复杂得多。如果我们收到一个损坏的ACK，我们就忽略它。如果这个ackNo（ACK序号）属于其中一个待处理的帧，我们就用一个循环来清除缓冲区并将窗口的左墙移到右边。超时事件同样更复杂。我们首先启动一个新的定时器，然后我们重发所有的待处理帧。

算法11.8是在回退N帧自动重发请求协议中接收方算法。

算法11.8 回退N帧自动重发请求接收方算法

```

1  Rn = 0;
2
3  while (true) //不断重复
4  {
5      WaitForEvent();
6
7      if(Event(ArrivalNotification)) //数据帧到达
8      {
9          Receive(Frame);
10         if(corrupted(Frame))
11             Sleep();
12         if(seqNo == Rn) //如果为期望帧
13         {
14             DeliverData(); //传送数据
15             Rn = Rn + 1; //滑动窗口
16             SendACK(Rn);
17         }
18     }
19 }

```

分析 这个算法是简单的。我们忽略了损坏的帧或者失序的帧。如果一个帧与一个被期待的序列号一起到达，我们就传送数据，更新R<sub>n</sub>的值，并发送一个ACK，其ackNo指明了下一个被期待的帧。

### 例11.6

图11.6描述了一个回退N帧的例子。这个例子中前向通道是可靠的，而反向通道则不可靠。没有数据帧丢失，但是有一些ACK被延迟了并且有一个被丢失了。这个例子同样指出了再确



认被延迟或丢失的情况下,累积的确认能有所帮助。

在初始化后,有7个发送方事件。请求事件由来自网络层的数据触发,到达事件由来自物理层的确认触发。这里没有超时事件,因为所有待处理的帧在超时前都被确认了。注意,虽然ACK2丢失了,ACK3能起到ACK2和ACK3两者的作用。

这里有4个接收方事件,所有的事件由来自物理层的帧的到达触发。

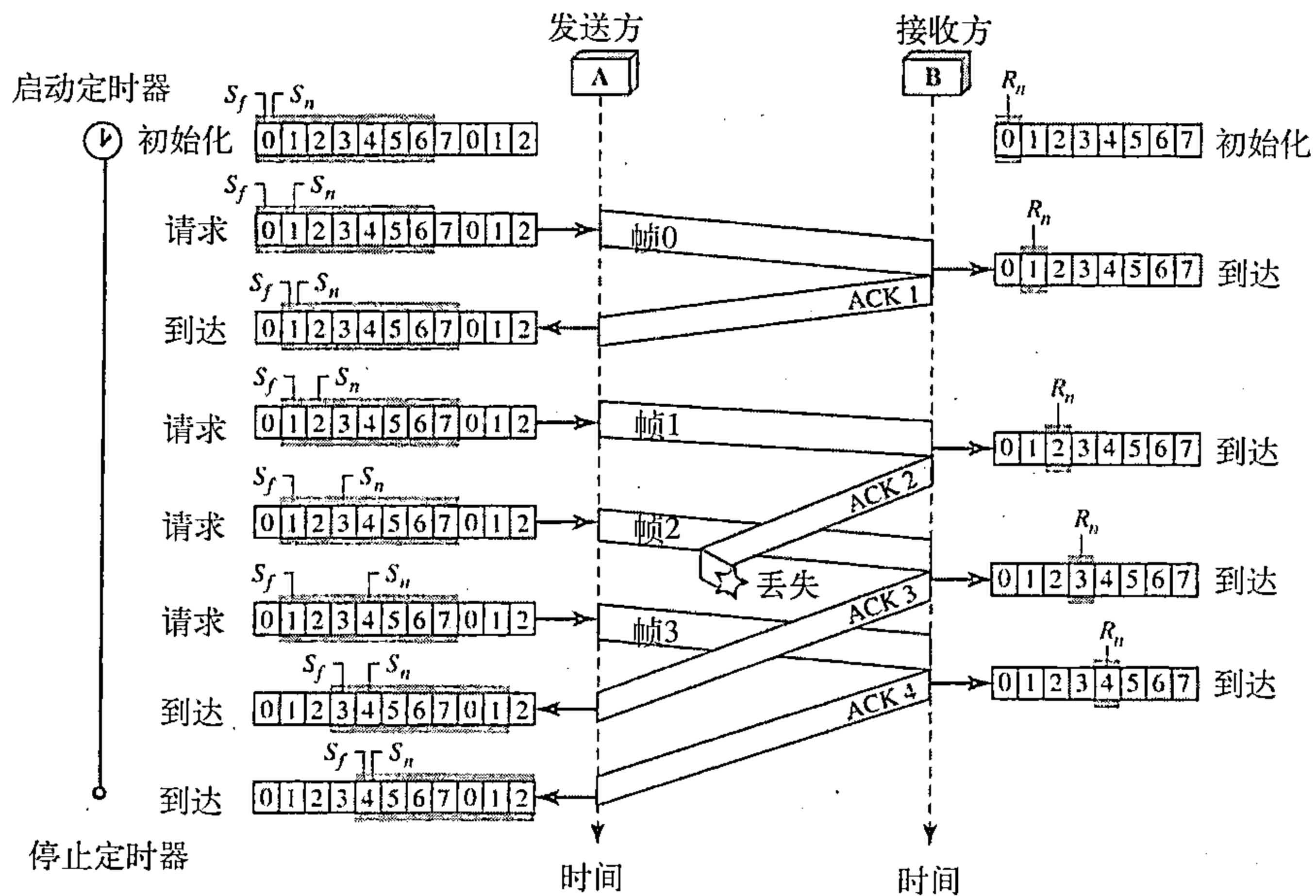


图11.16 例11.6的流量图

### 例11.7

图11.7说明了如果一个帧丢失了,情况会怎样。帧0、1、2和3被发送了,而帧1丢失了。接收方收到了帧2和3,但是它们被舍弃了因为被收到时它们失序了(被期待的是帧1)。发送方没有收到任何与帧1、2和3相关的确认。定时器最终到时,发送方再发送所有待处理的帧(1、2和3),因为它不知道哪里出错了。注意,帧1、2和3的重发是对一个单一事件的响应。当发送方对这一事件响应时,它不能在接受其他事件的触发。这就意味着,当ACK2到达时,发送方仍然在忙着发送帧3。物理层必须等待直到这一事件被完成,而数据链路层重新回到休眠状态。我们画了一条垂线来显示延迟。ACK3发生了同样的状况,但当ACK3到达时,发送方正忙着应答ACK2。当ACK4到达时,同样的状况又发生了。注意在第二个定时器到时前,所有待处理的帧已经都被发送了且定时器停止了。

### 回退N帧ARQ与停止等待ARQ的比较

读者能发现回退N帧的ARQ与停止等待ARQ有相同之处。我们可以认为停止等待ARQ协议实际上是回退N帧ARQ协议的一种,即当只有2个序列号而窗口的大小为1时。换言之,就是 $m=1$ ,  $2^m-1=1$ 。在回退N帧的ARQ中,条件是模 $2^m$ ,而在停止等待ARQ中,条件是2,即 $m=1$ 时,与 $2^m$ 一样。

停止等待ARQ是一种窗口大小为1的特殊的回退N步帧ARQ协议。



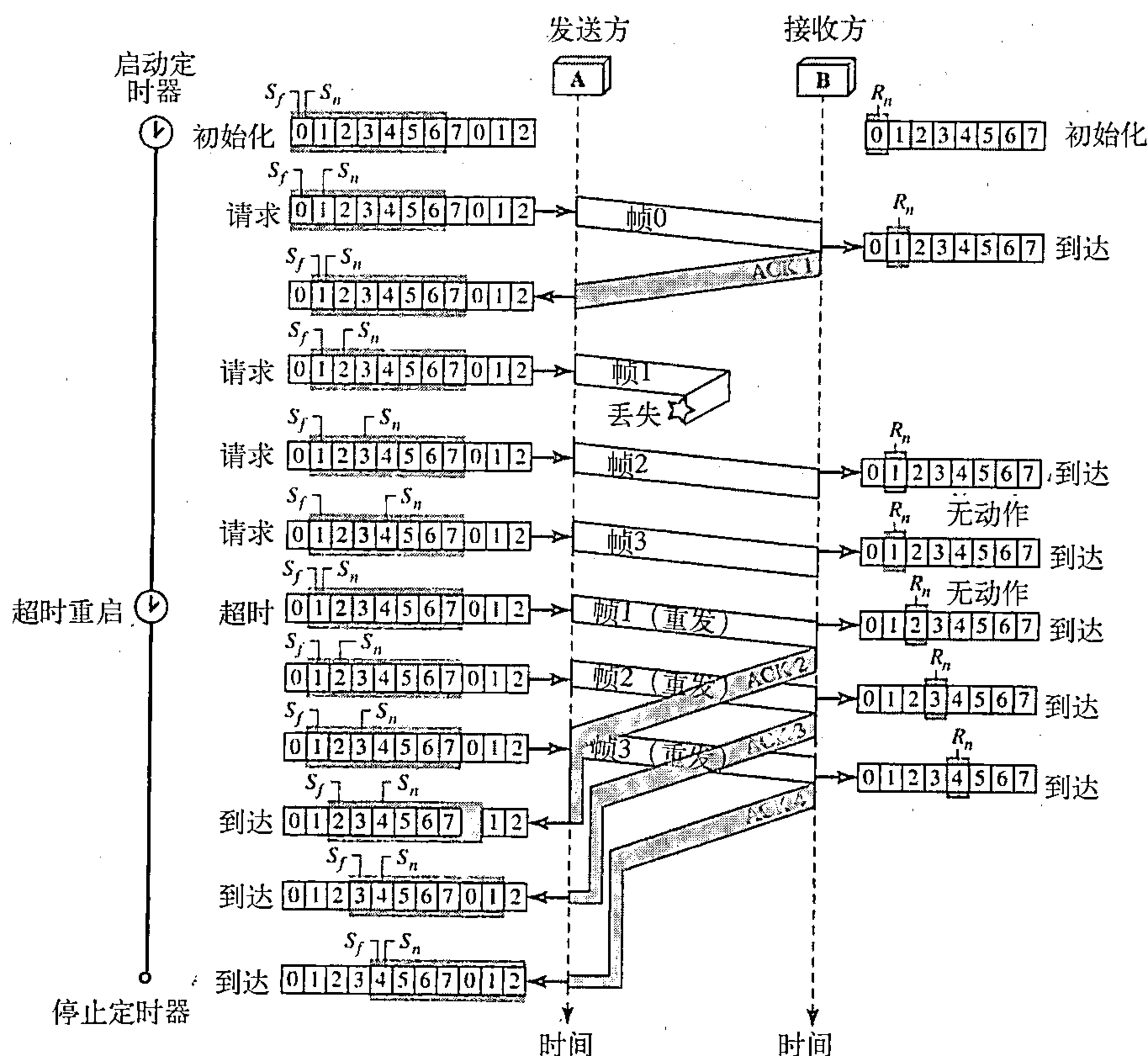


图11.17 例11.7的流量图

### 11.5.3 选择性重复ARQ

回退 $N$ 帧ARQ协议简化了接收方的程序。接收方只持有一个变量，且没有必要缓冲失序的帧，它们只是被简单地丢弃。然而，这个协议在有噪声链路中十分低效。在有噪声链路中，一个帧被损坏的可能性更大，这就意味着要重发多个帧。重发占用了带宽并减慢了传输的速度。对于有噪声链路而言，有另一种机制可以在一个帧被损坏时不必重发 $N$ 个帧。这个机制就称为选择性重复ARQ (Selective Repeat ARQ)。它对有噪声链路更为有效，但是接收方的程序也更复杂。

#### 窗口

选择性重复协议同样使用两个窗口：发送窗口和接收窗口。但是，这些窗口与回退 $N$ 帧协议中的窗口有所不同。首先，窗口的尺度更小，为 $2^{m-1}$ 。原因我们在后面解释。第二，接收窗口与发送窗口的大小一样。

发送窗口最大只能是 $2^{m-1}$ 。例如，如果 $m=4$ ，序号为0到15，但是窗口的大小只能是8（而在回退 $N$ 帧协议中，它是15）。窗口尺度变小意味着对管道填充的有效性减少，但事实上副本帧会少很多，能使这点得到补偿。该协议使用了与回退 $N$ 帧协议相同的变量。图11.18显示了选择性重复协议中的发送窗口来强调窗口的大小。将其与图11.12进行比较。

在选择性重复协议中的接收窗口与回退 $N$ 帧的接收窗口完全不同。首先，接收窗口的大小和发送窗口的一样（ $2^{m-1}$ ）。选择性重复协议允许与接收窗口大小一样多的帧乱序到达，并被保留直到有一组有序帧能交付给网络层。因为发送窗口的大小和接收窗口的一样，所有被发送的帧能乱序到达并被保存直到它们被传输。然而，我们需要指出的是接收方从不会将无序